Democratizing Robot-Use Agents: A Unified Harness with Open Models

Jiarui Yuan*, Yuxin Zuo*†, Xiaoyu Huang*†, Wenbo Zhang*, Kwanyoung Park*, Junlin Yang*, Tianxiang Zhang*, Ruoqu Chen*, Yutao Ouyang*, Bingxiang He*, Xingtai Lv*, Hongjin Wu*, Jiajun Liu, Jiahao Zhao, Tianwei Luo, Haozhan Li, Cheng Qian, Jiaxin Ge, Huan-ang Gao, Kaisen Yang, Jiaqi Deng, Qixin Xu, Siqiao Huang, Bowen Zhou, Mengdi Xu, Zhiyuan Liu, Pieter Abbeel, Ning Ding

Tsinghua University UC, Berkeley

*Core Contributor. †Project Lead.

Blog | Code | Website | Documentation

arXiv:2611.01560v1 [cs.RO] 1 Nov 2026

Abstract

Robot-use agents (RUAs) have shown promising zero-shot manipulation capabilities, but reliance on frontier, proprietary models limits research into further training. To democratize RUA research, we introduce uni-robot-agent, a unified harness for competitive open-weight robot use that supports further training. Without privileged inputs, an out-of-the-box Qwen3.8-Flash-Next achieves a RoboDojo Score of 31.39, surpassing the published GPT-6 Astra–RoboProbe result of 28.97 and many VLA/WAM baselines, and performs competitively on 19 real-world tasks. Our analysis across model generations, families, and sizes links effective robot use to four general capabilities, suggesting that future RUA training should develop them jointly.

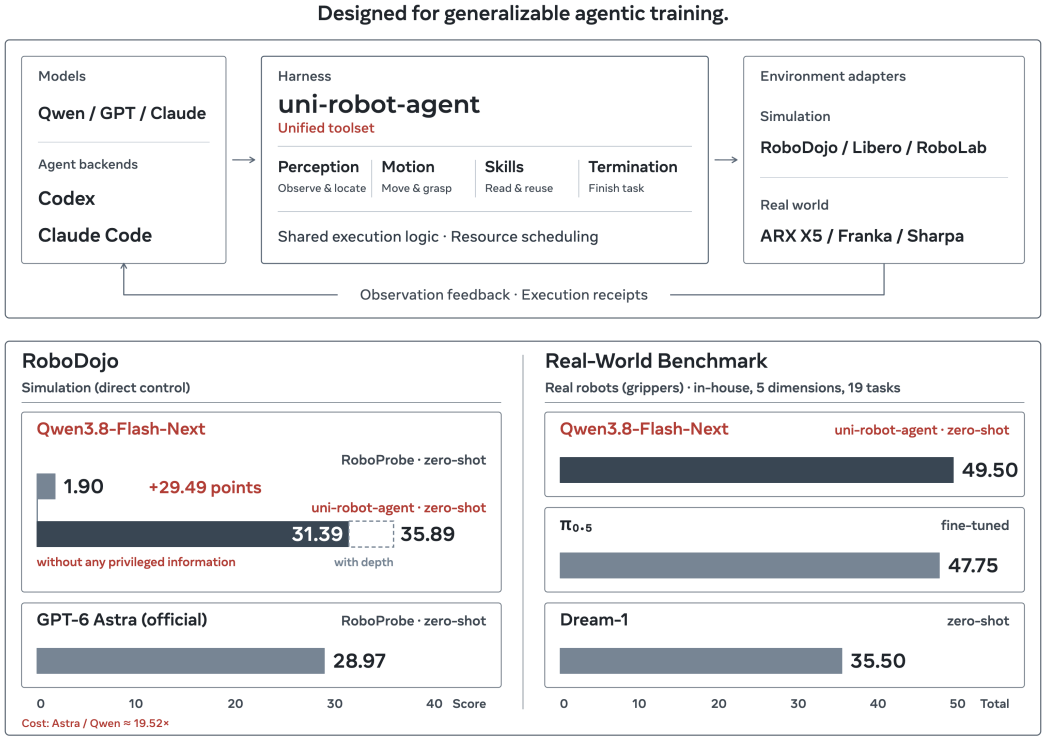


Figure 1 | **uni-robot-agent: results and system overview.** Top: shared tools, execution, and feedback connect models to environment adapters, supporting generalizable agentic training across environments. Bottom: Simulation and real-world scores.

Contents

1 Introduction **4**

2 uni-robot-agent: A Unified and Modular Harness **5**

2.1 System Overview 5

2.2 Unified Interaction Interface 6

2.3 Efficient Execution 7

2.4 Interaction Without Privileged Inputs 7

2.5 Key Design Choices 7

3 Can Open Models Use Robots in Simulation and the Real World? **9**

3.1 Simulation Evaluation 9

3.2 Real-World Evaluation 11

4 What Capabilities Enable Robot-Use Agents? **11**

4.1 Setup 11

4.2 Performance Across Generations, Families, and Model Sizes 12

4.3 Four Major Failure Modes 13

4.4 Relationship to General Model Capabilities 15

5 Analysis on Privileged Information **15**

5.1 RQ1: How Does Depth Affect Robot Use? 16

5.2 RQ2: Is skill library important for uni-robot-agent? 17

6 Analysis on Cost and Speed **17**

6.1 Token Usage and API Cost 17

6.2 Execution Efficiency 19

7 Related Work **20**

8 Conclusion **21**

A Experimental Setup and Reproducibility Settings **25**

B Real Eval Benchmark Details **25**

B.1 Hardware and Baselines 25

B.2 Training Task Catalogue and Collection Protocol 25
B.3 Evaluation Dimensions 26
B.4 Per-Task Results 28
B.5 Scoring Protocol 28

C API Cost Estimation **28**

D Monitored Motion Execution **29**

E Spatial Grounding Through Agent-Written Code **32**
E.1 Recovering Tube Positions Across Views 32
E.2 Exploring Geometry Before Moving a Cup 33

F Planning During Physical Execution **33**
F.1 From Regrasping to Supported Sliding 34
F.2 Preparing Reorientation During Transport 35

1. Introduction

Recent breakthroughs in frontier foundation models (e.g., GPT-6 Astra (OpenAI, 2026b), Opus 5.5 (Anthropic, 2026)) have shown surprising zero-shot performance in robotic manipulation, highlighting robot-use agents (RUAs) (Isola, 2026) as a promising approach for robotics (Chen et al., 2026b; Qu et al., 2026; Zhang et al., 2026b). This approach effectively transfers the models' generic problem-solving capabilities, such as task decomposition and failure recovery, which are hard to learn from limited robotics data alone, yet extremely useful for robotics (Chu et al., 2026; Isola, 2026; Jia et al., 2026).

However, because these frontier foundation models are proprietary, research on RUAs has been limited to using the models as-is, such as harness design (Li et al., 2026; Si et al., 2026; Zhang et al., 2026b). Without access to their weights, researchers cannot directly explore how further training of these models can advance RUA capabilities. Moreover, API dependence makes large-scale interaction costly (OpenAI, 2026b) and restricts model-level optimization, such as efficient caching, despite inference speed being a major bottleneck for RUA (Isola, 2026).

The main challenge in adapting open-weight models for robot use is the capability gap between open-weight and closed-weight foundation models. Robot use demands strong spatial understanding and agentic capabilities, which existing open-weight models often lack. Consequently, prior work has focused on ad-hoc improvements, including leveraging privileged information such as ground-truth simulator states or oracle task-success feedback (Fu et al., 2026; Zhang et al., 2026c), or incorporating task-specific skills (Lu et al., 2026). Before attributing these limitations to the models themselves and pursuing ad-hoc improvements, we argue that we must first establish a generalizable and effective robot-use harness to systematically understand and maximize their existing capabilities, laying the foundation for future RUA training.

In this paper, we take a first step toward democratizing robot-use agent research by laying this foundation for both competitive open-weight agents and future robot-specific training. We introduce uni-robot-agent, a general-purpose RUA harness. It provides **Unified** tool interfaces and feedback formats across models, embodiments, tasks, and environments, supporting joint training on simulation and real-world interaction data. For future large-scale training, we emphasize **Efficiency** by overlapping model reasoning with tool execution to accelerate interaction, and **Modularity** through interchangeable agent runtimes, tools, and robot backends. Together, these designs provide a scalable, extensible foundation for RUA research.

The proposed uni-robot-agent enables competitive manipulation performance with open-weight models, including Qwen and DeepSeek, in both simulation and physical environments. A key finding is that well-structured information, such as camera calibration and robot descriptions, together with reliable tool interfaces that provide informative feedback, are **sufficient** for effective robot use. As a result, **without privileged inputs**, Qwen3.8-Flash-Next with uni-robot-agent achieves a Score of **31.39**, surpassing GPT-6 Astra's result (**28.97**) on RoboDojo (Chen et al., 2026a) from the RoboDojo team (Zhang et al., 2026a). To the best of our knowledge, this is the first system to outperform closed-weight foundation models on RoboDojo using open-weight models, without privileged informations or distillation from those closed-weight models. In real-world experiments on 19 tabletop manipulation tasks, Qwen3.8-Flash-Next with uni-robot-agent achieves a Score of **49.50**, outperforming Dream-1 (**35.50**; Dream Labs, 2026) and matching fine-tuned $\pi_{0.5}$ (**47.75**; Physical Intelligence et al., 2025).

Having established the competitiveness of open-weight RUAs, we further compare models across generations, families, and sizes to investigate the capabilities underlying robot use. Trajectory analysis reveals four common failure patterns involving visual understanding, spatial

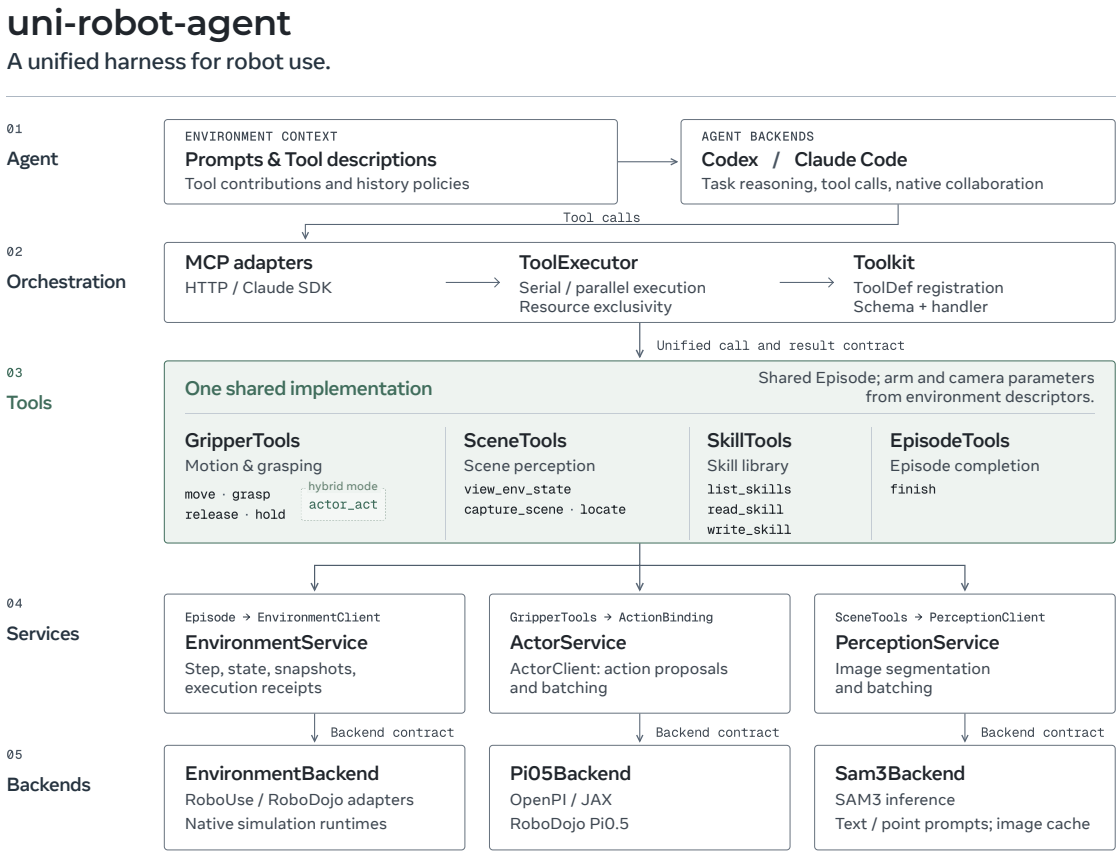


Figure 2 | **Five logical layers of uni-robot-agent.** Requests flow from agent runtimes through orchestration and shared tools to domain services and native backends. Context policies and concrete backends are replaceable components; arrows summarize logical interactions rather than individual network connections.

grounding, geometric coding, and embodied reasoning. Based on these findings, we posit that current robot-use capability emerges from a combination of these general-purpose capabilities and identify them as key areas for improvement in future RUA training. Finally, we provide an analysis of the effect of privileged information, cost, and execution efficiency.

2. uni-robot-agent: A Unified and Modular Harness

In this section, we introduce the key mechanisms of uni-robot-agent. We describe how the harness connects diverse agents, robot environments, and embodiments through shared tools and supports interaction based on information only available in real-world deployments.

2.1. System Overview

uni-robot-agent connects different agent runtimes to a shared toolkit for simulation and physical robots. The agent selects operations using task instructions and observations, while the harness executes them and returns feedback. We organize this interaction around four principles:

- **Unified.** Shared tools give operations and feedback a common meaning across environments, with adapters handling native APIs (Section 2.2). The resulting interaction records can also be combined for subsequent training.
- **Efficient.** Reasoning should overlap physical execution, while operations that share a resource retain their required order (Section 2.3).
- **Realistic.** Agents and tools should rely on information available in physical deployments, without simulator-provided depth or task-verifier feedback (Section 2.4).
- **Modular.** Explicit interfaces should allow researchers to replace runtimes, tools, and perception models independently. We introduce this architecture below.

These principles guide how we organize the system, but reliable interaction also requires care in how observations, actions, and feedback are handled. We first describe the architecture, then examine these practical design choices in Section 2.5.

Modular Architecture. A tool call passes through five layers with explicit interfaces (Figure 2). The **agent layer** manages model interaction and context, passing tool requests to the **orchestration layer** for scheduling and dispatch. The **tool layer** implements shared robot operations and accesses environment, perception, and action-model capabilities through the **service layer**, which manages their runtime resources. The **backend layer** supplies the concrete simulator, robot, and model implementations used by these services. Results return through these interfaces to the agent, completing the interaction.

2.2. Unified Interaction Interface

Moving a gripper to a target pose should mean the same thing in simulation and on a physical robot, even when their control APIs differ. We give agents a shared set of tools for motion, perception, skills, and episode control, with common arguments, units, and feedback formats. Environment descriptions identify the available arms, cameras, and optional capabilities. This lets the agent use familiar operations while accounting for the robot and sensors available in each environment.

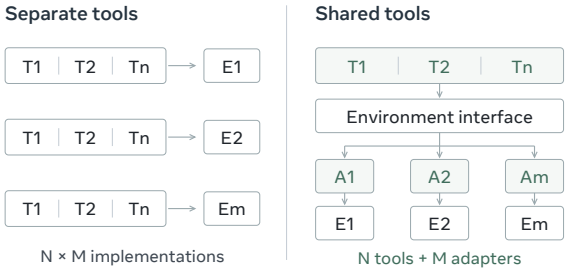


Figure 3 | **Tool reuse across environments.** Shared tools avoid $N \times M$ implementations. T: tool; A: adapter; E: environment.

To make these operations work across environments, shared tools access observations, robot state, and primitive actions through a common interface implemented by environment adapters. For example, the shared `move` tool generates intermediate targets and monitors progress; the adapter translates those targets into native commands and returns the resulting state. Adding an environment therefore requires an adapter and its configuration rather than a separate implementation of every tool. As Figure 3 illustrates, N tools can share their logic across M adapters, avoiding $N \times M$ tool–environment implementations. The common request and feedback formats also make the experience collected across environments easier to combine for subsequent training.

2.3. Efficient Execution

The orchestration layer determines how agent interaction proceeds while tools are running. Waiting for every movement to finish delays reasoning that could proceed independently, yet successive movements may still need to execute in order. We combine background tool calls with resource-aware scheduling to satisfy both requirements, forming a pipeline between agent reasoning and tool execution.

Two Levels of Concurrency. The agent need not wait for every tool, but its next call may still need to wait for a resource:

- **Between agent and tools.** A `background` call returns a job handle, allowing the agent to continue independent reasoning and retrieve the result later; an `immediate` call waits for completion. Decisions that depend on the result must still wait.
- **Among tool calls.** Calls sharing an exclusive resource execute in submission order; parallel calls run outside that queue. Two `move` calls on the same resource therefore run sequentially, while reading an existing snapshot can overlap movement.

Two-Stage Pipelining. These mechanisms enable a two-stage pipeline comprising agent reasoning and tool execution (Figure 4). The agent can submit successive calls while previously submitted calls execute in resource order. Reasoning and execution thus overlap even when the tools themselves execute sequentially. When observations or execution feedback require a change of plan, the agent can use `flush_queue` to cancel calls that have not started for a specified resource. Where supported, a single model response can submit multiple tool requests, and a batched wait can retrieve multiple results.

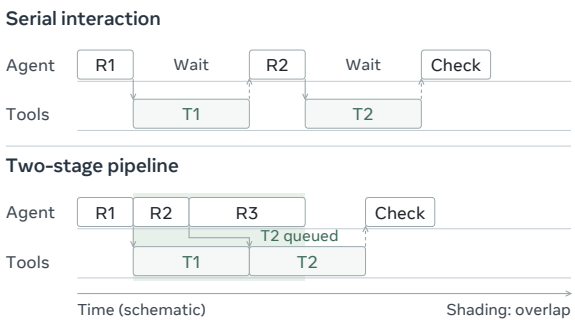


Figure 4 | **Serial interaction and pipelining.** Reasoning (R) overlaps sequential tools (T); T_2 queues while T_1 runs. Solid/dashed arrows show submissions/results; shading marks overlap. Timings are schematic.

2.4. Interaction Without Privileged Inputs

Efficient execution alone does not ensure that simulation performance carries over to physical robots: the information supporting each action also matters. Existing systems provide depth-based geometry and completion checks (Li et al., 2026), or construct object models to maintain a virtual scene (Wang et al., 2026). In physical settings, depth measurements can vary or be missing even in a static scene, and simulator task-verifier feedback is unavailable. These differences motivate testing whether agents can infer spatial relationships and assess progress from more limited observations.

2.5. Key Design Choices

Effective robot use requires careful design of how observations reach the agent, how commands are executed, and how outcomes inform subsequent decisions. We introduce six practical design

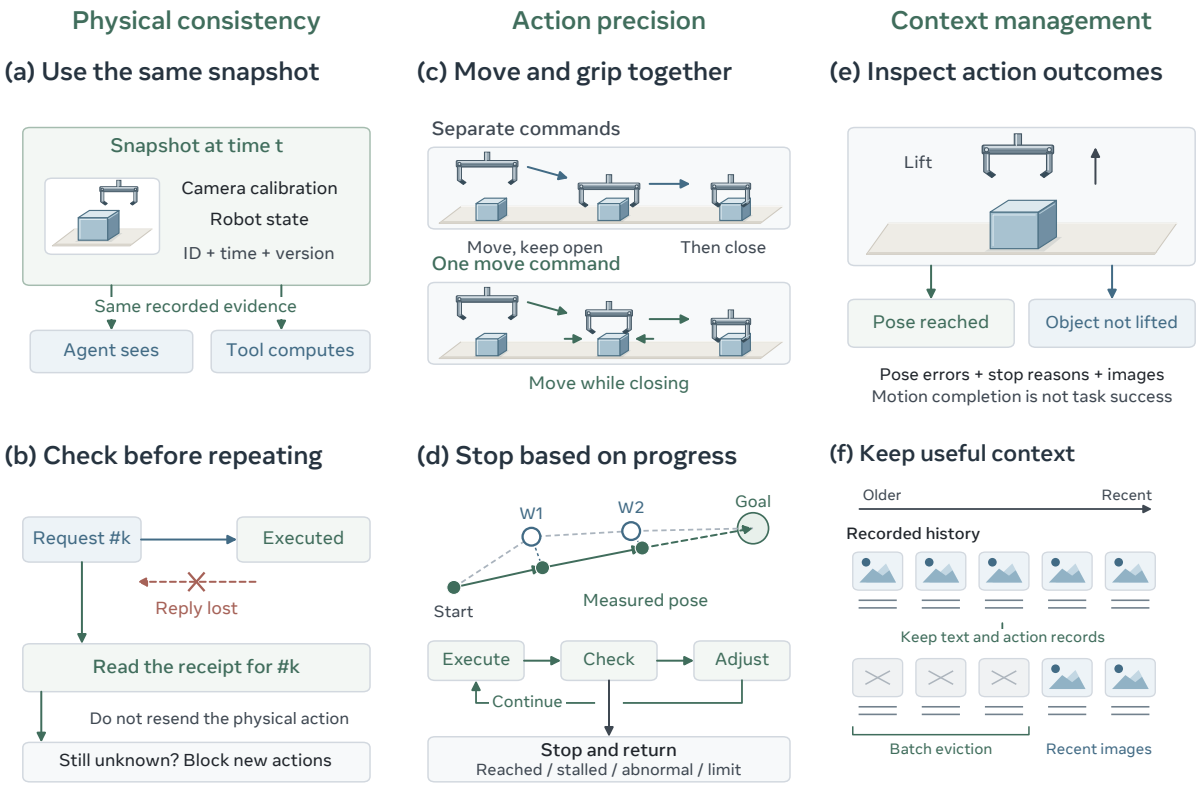


Figure 5 | **Key design choices.** Each column pairs two designs for physical consistency, action precision, or context management.

choices spanning observation and execution consistency, coordinated motion control, progress monitoring, actionable feedback, and visual context management.

Keeping Observations and State Consistent. When a robot moves after an image is captured, interpreting a selected pixel using a newer camera pose mixes two physical states. We bind images, camera calibration, and robot state to an immutable snapshot and use that snapshot for localization (Figure 5a). The agent and tools thus work from the same recorded state.

Preventing Repeated Actions After Communication Failures. The agent must also know which actions have occurred: a missing reply may mean lost feedback, so retrying could repeat a physical action. We record request identities and execution status to reject duplicates (Figure 5b). If completion is uncertain, further actions are blocked while the session attempts to recover the execution record; unresolved requests remain explicitly marked as uncertain.

Coordinating Motion and Gripper Control. Separate movement and gripper commands prevent the agent from adjusting the opening during a movement. Our `move` tool accepts a target pose and an optional gripper opening together, allowing the arm and gripper to act simultaneously (Figure 5c).

Detecting When Motion Should Stop. During execution, a fixed step count may stop motion too early or waste steps on an unreachable target. Our `move` tool generates intermediate targets

from measured poses and checks progress after each step (Figure 5d). This lets the tool adjust motion locally, stop on arrival, and return control when motion stalls or reaches an execution limit. Appendix D gives the algorithm and stopping criteria.

Making Action Outcomes Diagnosable. Reaching a target pose does not establish that an object was grasped or placed successfully. We return numerical motion feedback, including pose errors and stopping reasons, alongside images of the outcome (Figure 5e). The numbers distinguish a small residual error from rejected or stalled motion; the images show the effect on the object. Together, they help the agent decide whether to refine the motion or change its manipulation strategy.

Managing Growing Visual Histories. This feedback accumulates, but old images may lose relevance while task instructions and action records remain useful. Our sliding window replaces old images with text placeholders, preserving recent images and the surrounding text (Figure 5f). We evict images in batches to keep the prompt prefix stable between evictions and support cache reuse. If the context still exceeds budget, we compact the history with the same model that we use for the policy, using the compaction prompt of the codex CLI (OpenAI, 2026a).

3. Can Open Models Use Robots in Simulation and the Real World?

Effective robot use has largely been the preserve of frontier, closed-source models. In this work, however, we show that open-weight models can also achieve competitive performance in simulation and on physical robots. In particular, on RoboDojo, Qwen3.8-Flash-Next surpasses most fine-tuned VLA and WAM baselines in a zero-shot setting without privileged information, and also the GPT-6 Astra's result reported by the RoboDojo team. On real robots, a less-explored setting in prior works, open-weight models also achieve competitive performance, showing their real-world applicability. These results highlight the viability of open-source robot-use agents and the importance of a properly designed harness for supporting these models.

3.1. Simulation Evaluation

3.1.1. Setup

We evaluate 42 RoboDojo (Chen et al., 2026a) tasks spanning generalization, memory, precision, long-horizon execution, and open-vocabulary instruction following. We report the success rate and the score (incorporating partial success, scaled to 0–100) following the RoboDojo's evaluation protocol. The result is averaged over three seeds with two layouts (six episodes per task, 252 episodes in total). The 12 generalization tasks pair equally weight standard and randomized layouts per seed to test robustness to object and scene changes. Following the official protocol, we average the result of each tasks within the dimension, and average those results over the five dimensions.

No Privileged Inputs. Our simulation uses the inputs of realistic robotics applications, providing only RGB images and camera calibration, robot proprioception and descriptions, and task instructions. We do not allow access to ground-truth depth from simulators, CAD assets of the object, hidden scene annotations, task-verifier feedback, and external web access. We also manually check that the models did not made any invalid access to the prohibited information throughout the experiment.

Table 1 | **Results on RoboDojo.** Each entry reports SR/Score, where success rate (SR) is a percentage and Score is on a 0–100 scale. Our uni-robot-agent configurations exclude task-verifier feedback. Depth-enabled results are shown in gray for reference; our primary results do not use depth. Avg. is the unweighted mean of each metric over the five capability dimensions. †GAP (Hybrid) incorporates $\pi_{0.5}$.

Model	**Harness**	Gen.	Prec.	Long-Hor.	Memory	Open	**Avg.↑**
				VLA & WAM			
$\pi_{0.5}$	–	8.17/13.38	5.50/12.40	14.67/23.54	4.67/5.89	1.67/1.98	6.93/11.44
Hy-Embodied-0.5-VLA	–	8.39/11.78	8.00/13.81	14.92/25.74	12.11/13.37	0.58/0.65	8.80/13.07
OpenWAM-α	–	14.83/20.71	9.25/18.45	25.33/34.93	9.11/10.41	1.08/1.41	11.92/17.18
Xiaomi-Robotics-1	–	17.00/23.54	18.83/26.69	23.67/38.39	6.56/7.81	3.58/3.94	13.93/20.07
				Agent			
GPT-6 Astra	RoboProbe	30.50/33.36	4.00/12.65	8.25/21.45	38.67/43.04	31.00/34.36	22.48/28.97
GPT-6 Astra	**uni-robot-agent**	31.94/37.15	20.83/28.75	39.58/50.31	80.56/80.56	60.42/62.71	46.67/51.90
GPT-6 Astra	uni-robot-agent + depth	43.06/47.85	29.17/36.04	52.08/60.31	75.00/75.42	62.50/64.17	52.36/56.76
Qwen3.8-Flash-Next	RoboProbe	0.00/0.35	0.00/0.83	0.00/3.75	0.00/0.00	4.17/4.58	0.83/1.90
Qwen3.8-Flash-Next	RPent	5.56/9.24	4.17/8.02	4.17/12.40	5.56/5.56	2.08/3.54	4.31/7.75
Qwen3.8-Flash-Next	GAP (Hybrid)†	13.89/19.31	8.33/16.46	18.75/28.85	16.67/16.67	22.92/23.75	16.11/21.01
DeepSeek-V4.1-Flash	**uni-robot-agent**	16.67/20.35	6.25/16.25	4.17/14.79	36.11/38.33	43.75/45.83	21.39/27.11
DeepSeek-V4.1-Flash	uni-robot-agent + depth	18.06/21.88	12.50/19.79	10.42/25.31	41.67/45.56	37.50/38.54	24.03/30.22
Qwen3.8-Flash-Next	**uni-robot-agent**	**18.06/21.60**	**2.08/10.21**	**6.25/23.75**	**52.78/54.72**	**45.83/46.67**	**25.00/31.39**
Qwen3.8-Flash-Next	uni-robot-agent + depth	22.22/25.28	18.75/26.35	14.58/29.69	52.78/56.25	39.58/41.88	29.58/35.89

To contextualize open-weight model performance, we compare against official Astra–RoboProbe (Zhang et al., 2026a) results and representative VLA and WAM baselines from the RoboDojo leaderboard (RoboDojo Team, 2026). To compare the contribution of the harness, we compare Qwen3.8-Flash-Next across uni-robot-agent, GAP (Hybrid) (Su et al., 2026), RoboProbe, and RPent (Zhang et al., 2026c). Baseline configurations are detailed in Appendix A.

3.1.2. Results and Analysis

As shown in Table 1, Qwen3.8-Flash-Next with uni-robot-agent achieves **25.00% SR and 31.39 Score**, surpassing listed VLA/WAM baselines and GPT-6 Astra with RoboProbe (22.48% SR, 28.97 Score), despite being an open-weight model. DeepSeek-V4.1-Flash also outperforms the VLA/WAM baselines (**21.39% SR, 27.11 Score**). Moreover, switching to other harnesses results in a substantially degrades the performance (loss of 8.89% SR and 10.38 Score on the strongest baseline GAP). Together, these results demonstrate competitive robot use across two open-weight families and the importance of our harness on this performance.

Both open-weight or closed-source RUAs tend to achieve higher scores on memory and open-vocabulary tasks than the VLA and WAM baselines. On long-horizon tasks, however, Qwen and DeepSeek perform worse than Xiaomi-Robotics-1 and OpenWAM-α. We note that GPT-6 Astra equipped with uni-robot-agent performs the best, showing that the design choices of uni-robot-agent can also benefit closed-weight models too.

Table 2 | **Results on real-world evaluation.** Each capability is scored out of 20; Total is out of 100. Per-task results and scoring details are provided in Appendix B.

	VLA	**WAM**	**uni-robot-agent**	
Capability	$\pi_{0.5}$	Dream-1	GPT-6 Astra	Qwen3.8-Flash-Next
Spatial understanding	11.25	4.00	17.00	9.00
Object generalization	12.50	8.00	12.50	6.00
Instruction following	14.00	9.50	18.00	13.50
Compositional generalization	6.00	10.00	20.00	13.00
Unseen tasks	4.00	4.00	8.00	8.00
Total↑	47.75	35.50	**75.50**	49.50

3.2. Real-World Evaluation

3.2.1. Setup

We design an in-house benchmark with 19 tasks across five equally weighted dimensions: spatial understanding, object generalization, instruction following, compositional generalization, and unseen tasks. We evaluate Qwen3.8-Flash-Next and GPT-6 Astra with uni-robot-agent, alongside Dream-1 (Dream Labs, 2026) in a zero-shot setting, while $\pi_{0.5}$ (Physical Intelligence et al., 2025) is fine-tuned on 10 hours of demonstrations per task. All methods use the same dual-arm platform. We note that our agents additionally receive depth from Intel RealSense D405 cameras, while $\pi_{0.5}$ and Dream-1 use RGB only. Appendix B provides the task catalogue, scoring protocol, hardware, and baseline details.

3.2.2. Results and Analysis

As shown in Table 2, Qwen3.8-Flash-Next with uni-robot-agent matches the performance of fine-tuned $\pi_{0.5}$, and outperforms zero-shot Dream-1 in real world experiments. When applied to GPT-6 Astra, it outperforms other baselines with a large margin. Generally, RUAs perform better on compositional tasks and unseen tasks, while VLA/WAMs performing well on object generalization tasks. These results show that uni-robot-agent can be also applied to real world applications. Please refer to Table 4 for detailed per-task results.

4. What Capabilities Enable Robot-Use Agents?

The results in Section 3 establish the feasibility of robot use with open-weight models, but leave an important question: what enables these models to work? To investigate this, we examine three comparisons. First, we revisit earlier model generations, including Qwen3.5–3.7 and GPT-5.6. Second, we compare recent models from different families, including Qwen, DeepSeek, GLM, and GPT. Third, we examine Qwen models of different sizes within the same generation.

4.1. Setup

We evaluate the additional models on RoboDojo's 42 tasks and report SR and Score using the same aggregation as Section 3. We reuse the Section 3 results for Qwen3.8-Flash-Next, DeepSeek-V4.1-Flash, and GPT-6 Astra. The additional evaluations use one seed and one layout.

4.2. Performance Across Generations, Families, and Model Sizes



Figure 6 | **Robot-use performance across model generations, families, and sizes.** Success rate (top) and Score (bottom) on RoboDojo with uni-robot-agent. Robot use is neither exclusive to frontier models nor unique to the latest generation: GPT-5.6 Sol already achieves nontrivial performance, while Qwen3.8-27B succeeds where larger, earlier Qwen models fail. Shading indicates closed-source models.

Robot use predates the latest model generation. Figure 6 shows that GPT-5.6 Sol already achieves a nontrivial **19.17% SR and 24.04 Score**, while GPT-6.1 Sol reaches **37.50% SR and 45.63 Score**. Thus, the latest generation substantially improves robot-use performance, but these capabilities were already present in earlier models.

The pattern differs across model families. However, among the models evaluated, only the GPT series demonstrates successful robot use in generations preceding the latest one. For Qwen, verified task success is observed only in the 3.8 generation: Qwen3.5–3.7 Flash, Qwen3.6–3.7 Plus, and Qwen3.5-122B record zero SR, whereas Qwen3.8-27B and Qwen3.8-Flash-Next achieve **8.06%** and **25.00%**, respectively. More importantly, a recent model generation does not by itself guarantee task success: GLM-5.3 Flash shows 0% success rate as older Qwen models.

Model size matters less than training distribution. Within Qwen3.8, Flash-Next (125B) achieves higher SR and Score than the 27B model. Across generations, however, Qwen3.8-27B completes tasks while all earlier Qwen models evaluated, including Qwen3.5-122B, Qwen3.7 Plus (> 150B parameters), and Qwen3.6 Plus (> 150B parameters), record zero SR. We posit that training distributions differ substantially across generations and that these differences matter more for robot use than model size alone.

Taken together, these results suggest that robot use is neither exclusive to frontier models nor a capability that emerged uniformly with the latest generation. Earlier GPT models already

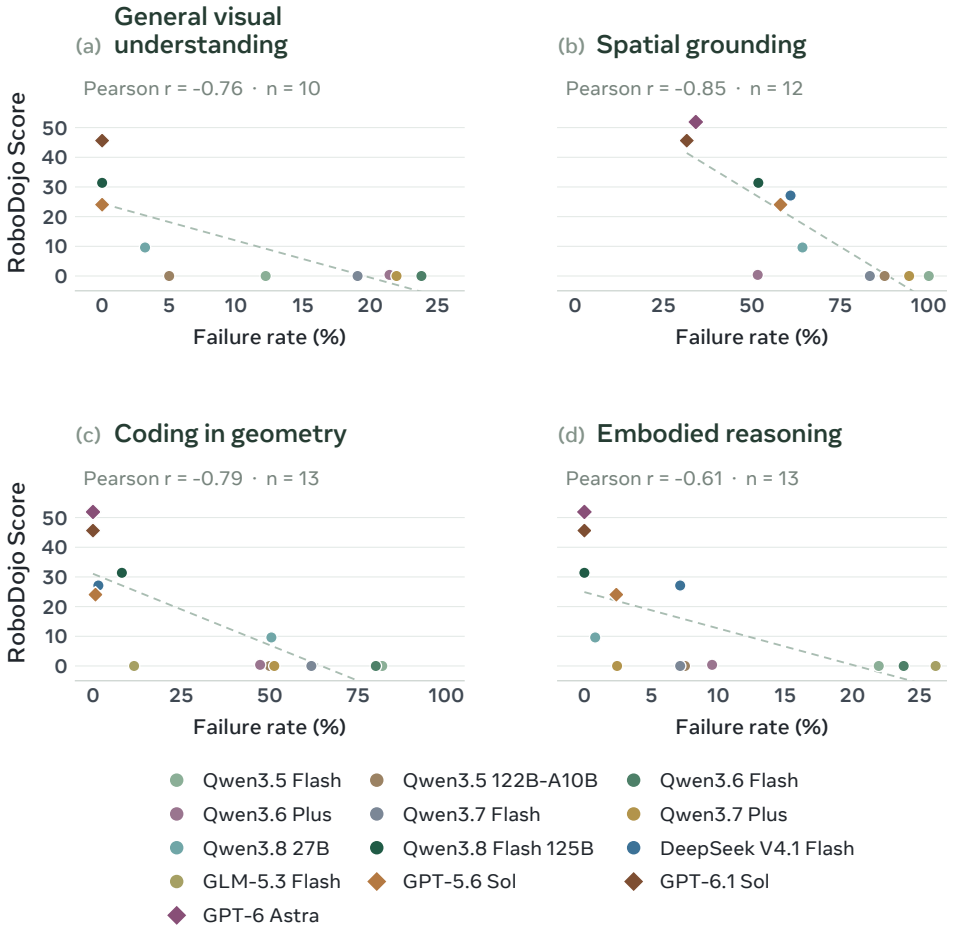


Figure 7 | **Four capabilities associated with effective robot use.** Correlation between each model's RoboDojo Score and its errors in (a) general visual understanding, (b) spatial grounding, (c) coding in geometry, and (d) embodied reasoning. Lower error rates across these dimensions are associated with higher task performance, suggesting that effective robot use draws on a combination of these capabilities

achieve nontrivial performance, while Qwen3.8-27B succeeds where larger, earlier Qwen models fail. Rather than model size or recency alone, these contrasting trajectories point to differences in the underlying capabilities (i.e. training distribution) developed across model families and generations. We next examine failure modes to identify which capabilities distinguish successful robot use from failure.

4.3. Four Major Failure Modes

Our trajectory analysis identifies four major failure types: *general visual understanding*, such as misinterpreting objects or action outcomes; *fine-grained pointing and spatial grounding*, such as selecting inconsistent corresponding points across views; *localization and pose computation code*, involving errors in geometric calculations, coordinate transformations, or orientation construction; and *embodied reasoning*, such as proposing physically inappropriate grasp poses. Figure 7 summarizes these four failure types across thirteen models.

General visual understanding. Perhaps the most straightforward reason for failures in earlier Qwen Flash and Plus models is basic visual misunderstanding. Such errors, including mistaking zero for nine or inverted cups for upright ones, occur in **12.2–23.8%** of trials. In contrast, these kinds of errors occur in **3.2%** of Qwen3.8-27B trials and none of the trials from more performant models such as Qwen3.8-Flash or GPT-5.6 Sol. These observations suggest that general visual understanding is closely associated with its performance.

Spatial grounding. In addition to coarse scene understanding, we find that models struggle to pinpoint the same physical feature across camera views. Selecting inconsistent pixels for the same feature leads to large reprojection errors, causing the triangulation tool to reject the result. We therefore use the fraction of calls rejected for this reason to measure difficulty in pinpointing corresponding features. Task performance tends to be negatively associated with this failure rate: Qwen3.8-Flash and DeepSeek have lower failure rates than models with zero task success, including Qwen3.5–3.7 Flash and Qwen3.7 Plus. GPT-6.1 Sol achieves the lowest spatial-grounding failure rate among the evaluated models (31.5%), but this substantial remaining error rate highlights precise spatial grounding as a common challenge for further improvement (Figure 7b).

Coding in geometry Coding skill is also closely correlated with task performance, as illustrated by geometry computations involving coordinate transformations and orientations. Earlier Qwen models frequently reverse coordinate transforms or mix quaternion conventions, with such errors affecting **47.3–81.8%** of executed calculations across the earlier Flash and Plus cohorts. In contrast, the error rates are **8.2%** for Qwen3.8-Flash, **1.5%** for DeepSeek, and **0.7%** for GPT-5.6 Sol, with no errors in GPT-6.1 Sol's calculations. The pattern suggests that reliable robot-specific computation is another capability associated with effective robot use (Figure 7c).

Embodied reasoning. Beyond perceiving and computing a target pose, robot use requires embodied reasoning to propose appropriate interactions, including grasp poses. Because this broad capability is difficult to measure, we restrict our analysis to pick-and-place and use GPT-6 Astra to assess whether proposed grasp poses are physically reasonable. We report the rate at which the model proposed unnatural or infeasible grasping poses and failed to correct them in these tasks (Figure 7d). To our surprise, even recent models can lack this capability: GLM repeatedly fails to reorient its wrist for grasping, contributing to many unsuccessful trials and a **26.2%** error rate. Such errors are less frequent in newer Qwen and GPT models, which more often select appropriate grasp orientations. However, grasp-pose proposal is only one instance of embodied reasoning. GPT-6 Astra still struggles with other skills, including bimanual coordination, where inappropriate interaction plans limit performance. Embodied reasoning therefore appears to improve across model generations, but remains a substantial bottleneck beyond this narrow grasping analysis.

In conclusion, these failure patterns suggest that effective robot use draws on a combination of visual understanding, spatial grounding, geometric coding, and embodied reasoning, rather than a single robot-specific capability. This suggests that general model capabilities provide a substantial foundation for robot use. We therefore argue that future RUA training should develop these capabilities jointly – for example, by using robot data to construct training tasks targeting each capability – rather than focusing solely on fitting action outputs.

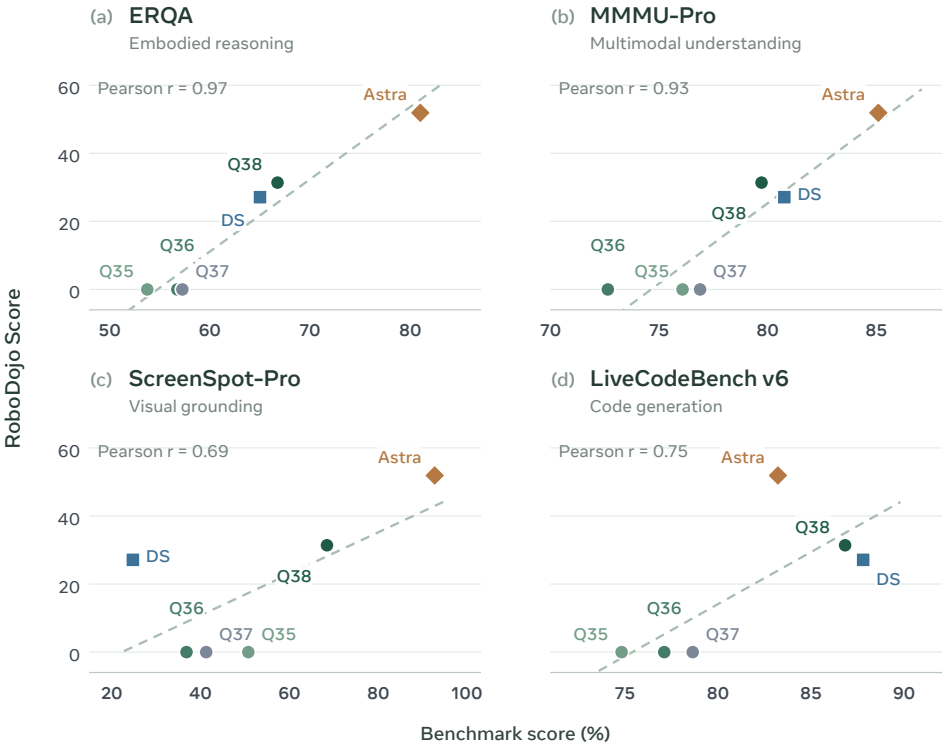


Figure 8 | **Robot-use performance and general model capabilities.** Each point denotes a model, with benchmark performance on the horizontal axis and RoboDojo Score on the vertical axis. Q35–Q38 denote Qwen Flash generations; DS denotes DeepSeek-V4.1-Flash-1017. Dashed lines show linear fits; Pearson correlations are computed across the six models.

4.4. Relationship to General Model Capabilities

Having identified four key capabilities underlying effective robot use through failure analysis, we next ask: To what extent can these capabilities be captured by existing foundation-model benchmarks?

To investigate this, we compare RoboDojo performance with scores on four corresponding benchmarks: MMMU-Pro (Yue et al., 2024) for general visual understanding, ERQA (Gemini Robotics Team et al., 2025) for embodied reasoning, ScreenSpot-Pro (Li et al., 2025) for spatial grounding, and LiveCodeBench v6 (Jain et al., 2024) for code generation (Figure 8). We observe positive correlations across all four benchmarks, with particularly strong correlations for embodied reasoning and general visual understanding, while spatial grounding exhibits a weaker correlation, potentially because ScreenSpot-Pro evaluates 2D screen-based grounding rather than the 3D spatial grounding required for robotic manipulation. These findings further support the role of general-purpose model capabilities in robot use, suggesting that advances in these capabilities may translate into improved robotic performance.

5. Analysis on Privileged Information

As discussed in Section 2.4, privileged inputs such as precise simulator-provided depth and oracle task-success feedback can assist robot interaction. Although uni-robot-agent operates without these inputs, it remains important to understand how much they affect robot-use performance. We first challenge whether depth, a commonly used source of geometric information,

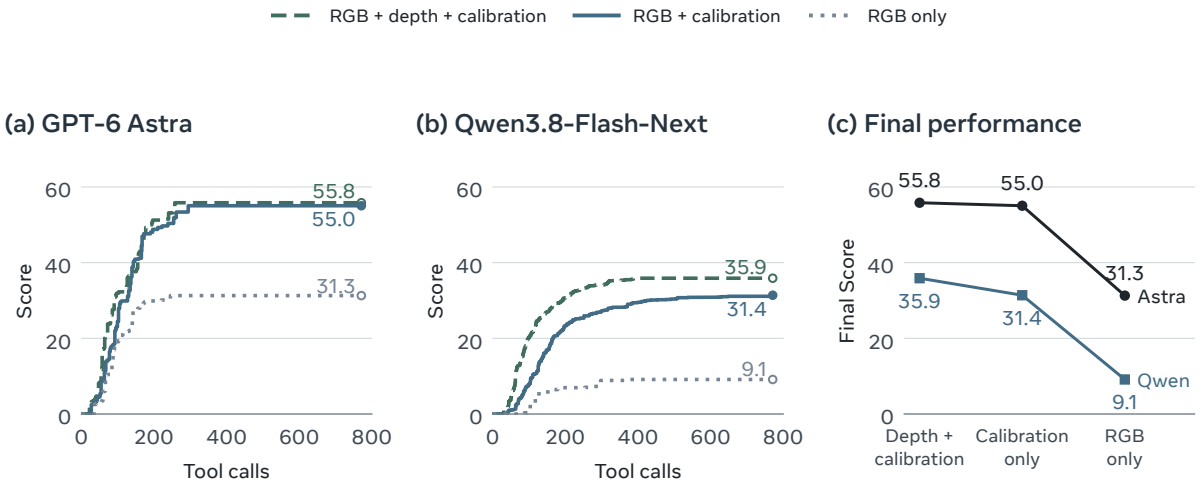


Figure 9 | **Robot use with less geometric information.** Recorded progress through 771 tool calls and final score across 42 tasks, with skills enabled.

is necessary for better performance (RQ1). Second, since skill libraries can encode privileged information or task-specific solutions, we ablate our skill library to test whether performance depends on such leak of privileged information (RQ2).

5.1. RQ1: How Does Depth Affect Robot Use?

We compare three levels of geometric information for depth: RGB images with depth and camera parameters, RGB images with camera parameters, and only RGB images. With depth and camera parameters, agent can infer 3D coordinates directly from the pixel. Having only camera parameters requires the agent to find the pixel of corresponding points in multiple cameras. Without either, the agents should infer the coordinates through spatial relationships of the objects in the image. We evaluate GPT-6 Astra and Qwen3.8-Flash-Next on the same 42 benchmark tasks. The evaluation uses three seeds and two layouts per task for Qwen and one seed and one layout per task for Astra due to limited evaluation costs.

Without depth, agents can still recover object positions with the camera parameters by triangulating the corresponding points across views. Evidently, Astra's score changes from **55.83** to **55.04**, and Qwen's score changes from **35.89** to **31.39**, retaining 98.6% and 87.5% of their depth-enabled scores, respectively (Figures 9a and 9b), reflecting a minimal loss in performance, especially with stronger model capabilities. The cases in Appendix E further shows that Qwen writes and executes its own geometry code to infer 3D coordinates, such as triangulating object positions across views and testing candidate table heights with the images.

Without camera parameters (only RGB images), where agents must infer 3D positions from abstract spatial relationships, Qwen's performance shows a sharp drop to **9.13**, while Astra shows a moderate decrease on its performance to **31.29** (Figure 9c).

In conclusion, we find that privileged depth information is **not necessary** for achieving strong robot-use performance as long as camera parameters are available. In fact, as model capabilities improve, models can more effectively infer 3D spatial relationships from visual observations, thereby benefiting less from such privileged inputs.

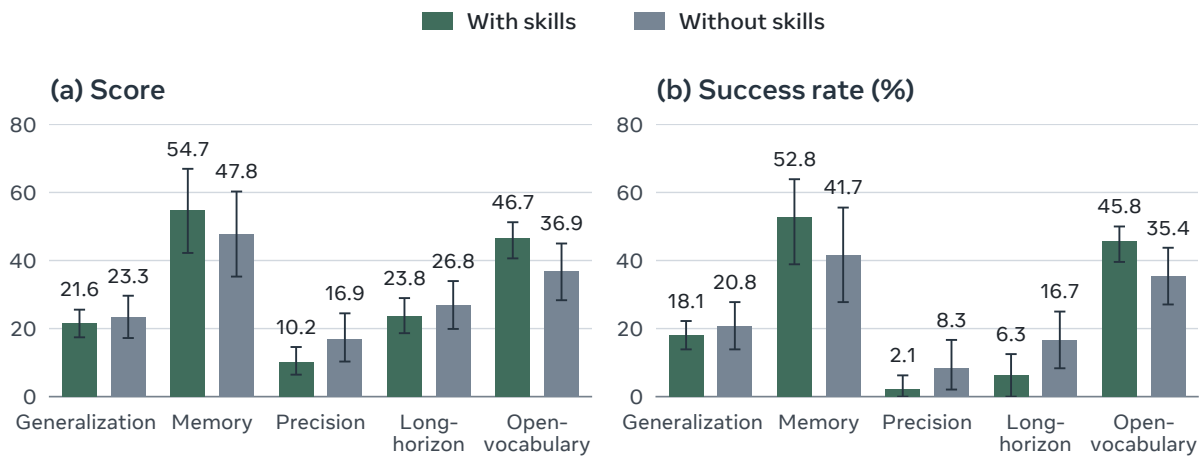


Figure 10 | **Effect of skill guidance on task performance.** Qwen3.8-Flash-Next with uni-robot-agent, evaluated with and without skills. Error bars indicate approximate 95% bootstrap confidence intervals.

5.2. RQ2: Is skill library important for uni-robot-agent?

Task-specific skill libraries may encode privileged information or predefined solutions that simplify robot tasks. We therefore examine whether the performance of uni-robot-agent depends on the knowledge provided by its skill library.

Using Qwen3.8-Flash-Next, we disable access to the robot skill library and skill writing (we retain task instructions, environment descriptions, and standard robot tools). Figure 10 shows a small aggregate performance gap between configurations with and without robot skills. The no-skill configuration reaches **30.31** score and **24.58%** SR, retaining **96.6%** of the skill-enabled score (31.39), with an SR gap of only 0.42 percentage points. These results show that uni-robot-agent maintains comparable performance without the skill library. The harness supports reusable skills, but its robot-use performance does not depend on the task-specific knowledge they provide.

6. Analysis on Cost and Speed

In this section, we focus on the time and resources required for uni-robot-agent across different models, and show that pipelining of uni-robot-agent substantially improves the execution time.

6.1. Token Usage and API Cost

A robot-use agent may spend many model calls and reasoning for locating objects, testing a grasp, or recovering from an unsuccessful action. We analyze the results from Section 3.1, examining how task performance relates to token usage, API cost, and progress during execution across different models. Reference API prices are provided in Appendix C.

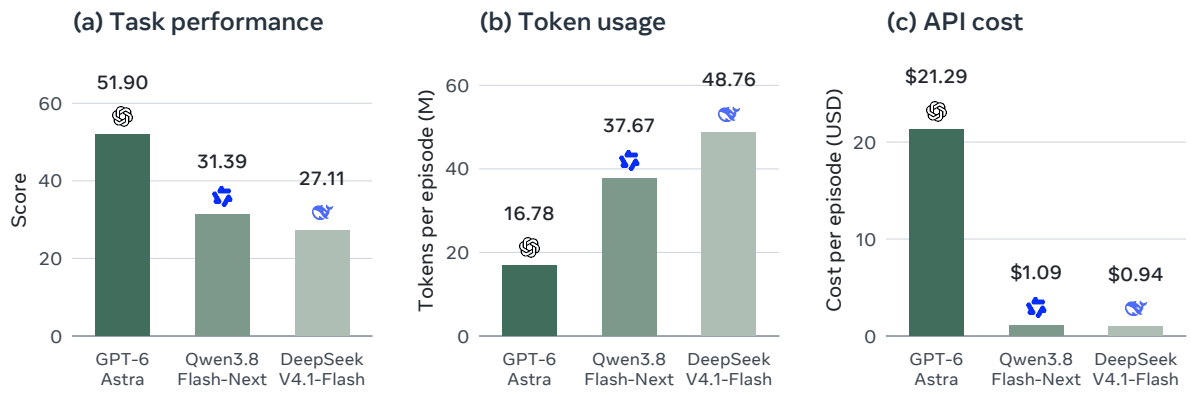


Figure 11 | **Model-level resource–performance trade-offs.** Astra, Qwen, and DeepSeek each use 252 episodes. Bars show score and mean resource usage per episode.

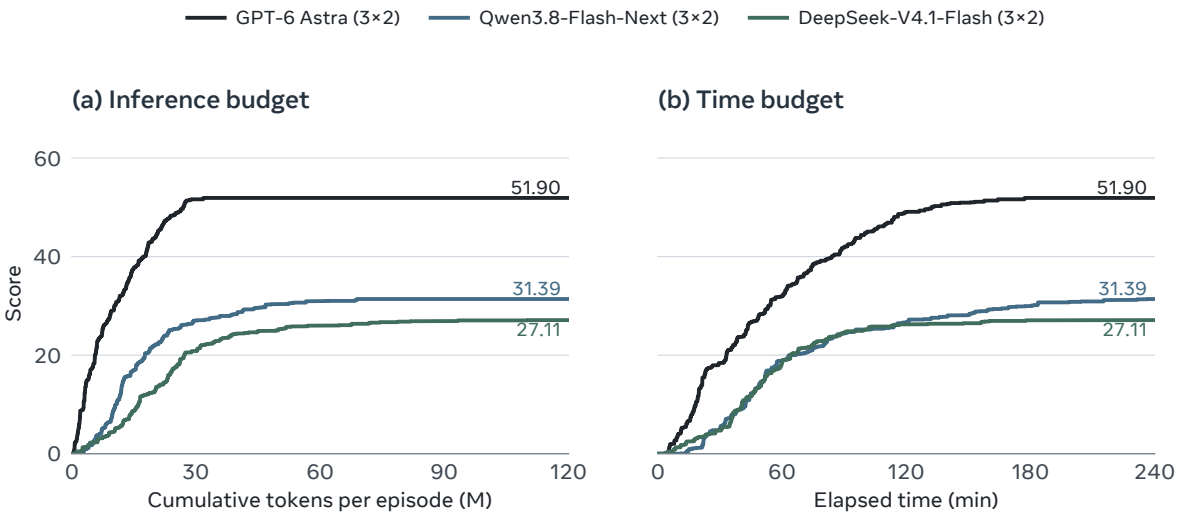


Figure 12 | **Task progress over token usage and time.** score is aggregated from recorded intermediate outcomes for the same model cohorts as Figure 11. Curves retain each episode's latest observed score.

Figure 11 shows that Qwen offers a favorable trade-off between the performance and the cost, reaching **31.39** score at $1.09 per episode. Astra achieves the highest score (**51.90**), but costs nearly 20x more ($21.29 per episode). This substantial difference of each model's cost highlights the value of advancing open-weight models for affordable robot-use agents.

Figure 12 shows that each models need substantially different amount of tokens to make progress. To reach a score of 25, Astra requires 7.4M tokens per episode, which is less than 23.5M for Qwen and 49.9M for DeepSeek. This trend also persists in terms of the wall-clock time, where Astra reaches a score of 25 after 42.6 minutes, while Qwen and DeepSeek take 95.5 and 99.2 minutes, respectively.

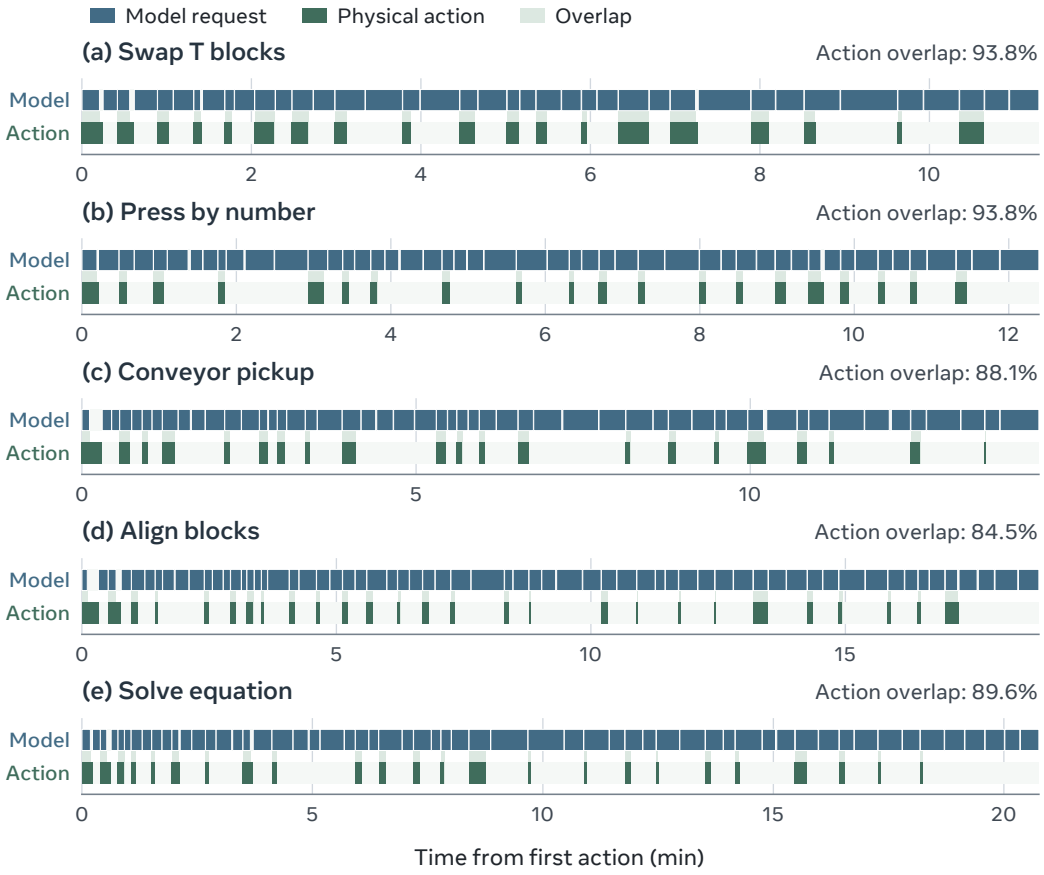


Figure 13 | **Model–action overlap in pipelined execution.** Blue and green bars show model requests and physical actions; shading marks overlap. Percentages indicate the fraction of action time overlapping model requests. Each panel starts at the first action and uses its own time range.

6.2. Execution Efficiency

The pipeline introduced in Section 2.3 allows model interaction to continue while physical actions is being executed. We select five tasks and compare pipelined and serial execution of uni-robot-agent using GPT-6 Astra with medium reasoning effort. Each task is evaluated once per configuration with the same seed and layout. We measure episode duration excluding simulator initialization and the overlap between model requests and physical tool execution.

Episode Duration. As shown in Figure 14, the pipelined runs have shorter episode durations on all five tasks, with observed reductions ranging from **4.8% to 73.7%** relative to serial execution. For example, `swap_T` takes 12.5 minutes with pipelining versus 34.7 minutes with serial execution, while `press_by_number` takes 13.7 versus 25.0 minutes.

Execution Overlap. We measure the fraction of physical execution time that overlaps with a model request. As shown in Figure 13, it ranges from **84.5% to 93.8%** in the pipelined episodes and is zero in all five serial episodes, showing that most of the physical execution time is pipelined with the model inference. Appendix F shows additional qualitative case studies.

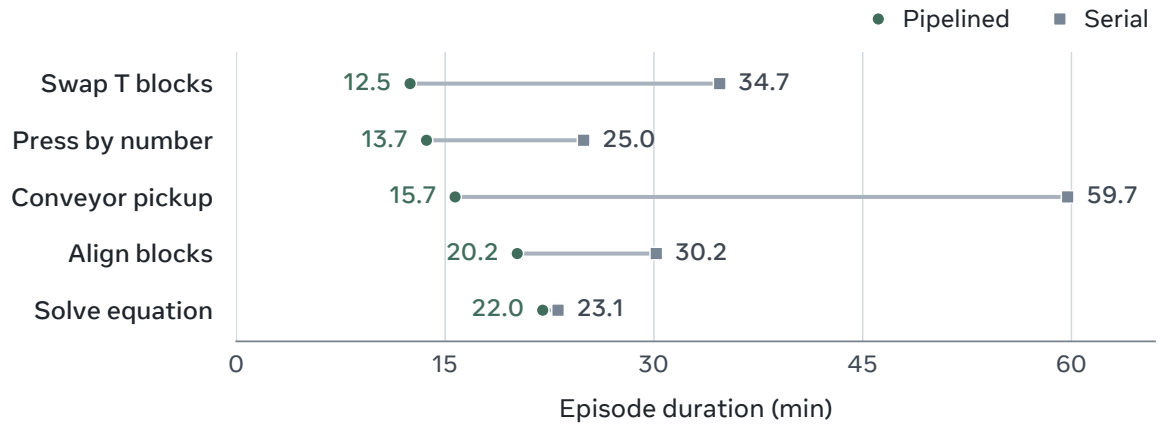


Figure 14 | **Episode duration on five selected tasks.** Pipelined and serial execution are compared using one episode per configuration and task, excluding simulator initialization.

7. Related Work

Vision-Language-Action and World-Action Models. Vision-language-action (VLA) models extend robot policies beyond task-specific demonstrations by transferring pretrained visual-language representations and learning from diverse robot data (Brohan et al., 2023; Kim et al., 2024). Training on heterogeneous sources has further improved generalization: $\pi_{0.5}$ performs manipulation in previously unseen environments by combining robot experience with semantic supervision and web data (Physical Intelligence et al., 2025). World-action models (WAMs) explore a complementary route through predictive modeling. Joint video and action generation enables DreamZero to generalize to unseen tasks and environments (Ye et al., 2026), while Cosmos Policy uses predicted future states and values to support test-time planning (Kim et al., 2026). These advances broaden where learned policies can operate and what behaviors they can perform. Reliable use of past observations and recovery from failures over extended interactions nevertheless remain important areas for further investigation.

Tool-Using Agents for Robot Control. Tool use allows language models to organize decisions around observations and feedback. In computer environments, agents operate graphical interfaces through repeated observation and action, combining visual grounding with planning and context management (Agashe et al., 2025; Anthropic, 2024; Wang et al., 2025). Robotics research has explored related ideas through skill selection and programmatic composition of perception and control APIs (Ahn et al., 2022; Liang et al., 2022). Environment feedback enables replanning (Huang et al., 2022), while spatial representations connect language instructions to physical motion (Huang et al., 2023). Together, these studies establish ways for general-purpose models to direct interaction through tools. In robotics, this interaction must also account for a world that changes while actions are running, making the coordination of observation, reasoning, and execution a central concern.

Robot-Use Agents and Harnesses. Recent studies show that general-purpose multimodal models can operate robots through tools without task-specific robot training (Isola, 2026; Zhang et al., 2026a). This has prompted efforts to design the harness through which a model observes the scene, issues actions, and receives feedback. RoboDawn and Robo-Harness K1 provide

motion and geometric perception tools (Guo et al., 2026; Li et al., 2026), while OpenRUA gives coding agents access to native ROS 2 interfaces through a terminal (Chu et al., 2026). Beyond these interfaces, Harness VLA and GAP investigate how agents can guide learned policies through memory-based planning and corrections to policy-generated actions (Su et al., 2026; Zhang et al., 2026c). ORA also demonstrates robot use with open-weight models, combining robot tools with environment information and context management (Yang et al., 2026). As robot use expands to more models and environments, reusing these components becomes an important system-design question. We develop uni-robot-agent around this need, separating shared robot tools from model runtimes and environment adapters so that changing one does not require rebuilding the others. We also coordinate tool execution and feedback to let agents continue reasoning while physical actions are underway.

8. Conclusion

We presented uni-robot-agent, a unified and modular harness with efficient execution and reliable tools for robot-use agents. Evaluations demonstrate strong performance with GPT-6 Astra in both simulation and real-world manipulation. More importantly, our results show that an open-weight model can also achieve strong robot-use performance: Qwen3.8-Flash-Next reaches a RoboDojo Score of 31.39, exceeding the published Astra–RoboProbe result of 28.97. These findings support further investigation of open-weight models that can be deployed locally and adapted through training on robot experience. Precise manipulation and reliable long-horizon execution remain challenges in our evaluations. By providing shared interaction interfaces, replaceable components, and structured execution feedback, uni-robot-agent offers a unified foundation for collecting robot interaction data, developing training methods, and evaluating their effects on agent behavior and task performance.

References

S. Agashe, K. Wong, V. Tu, J. Yang, A. Li, and X. E. Wang. Agent S2: A compositional generalist-specialist framework for computer use agents. *arXiv preprint arXiv:2504.00906*, 2025. doi: 10.48550/arXiv.2504.00906. URL https://arxiv.org/abs/2504.00906.

M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022. doi: 10.48550/arXiv.2204.01691. URL https://arxiv.org/abs/2204.01691.

Alibaba Cloud. Qwen3.8-Flash: Model information and pricing, 2026. URL https://help.aliyun.com/zh/model-studio/qwen3-8-flash. Beijing-region pricing; accessed October 25, 2026.

Anthropic. Developing a computer use model, Oct. 2024. URL https://www.anthropic.com/news/developing-computer-use. October 22, 2024. Accessed October 25, 2026.

Anthropic. Introducing Claude Opus 5.5, Sept. 2026. URL https://www.anthropic.com/claude-opus-5-5. Accessed October 27, 2026.

A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. doi: 10.48550/arXiv.2307.15818. URL https://arxiv.org/abs/2307.15818.

T. Chen, Y. Chen, Z. Li, J. Tang, K. Su, H. Lu, W. Wan, B. Chen, S. Liu, H. Yan, H. Su, Z. Dou, K. Wang, D. Zhang, Y. Liu, Y. Qin, Q. Liang, Q. Wu, Z. Lin, W. Lin, Y. Wang, M. He, T. Wu, R. Wu, J. Zhou, K.-C. Lei, H. Yu, Y. Ji, W. Jin, G. Lin, X. Li, Q. Xiong, R. Xu, Z. Li, W. Chai, E. Xie, Z. Wang, Y. Mu, H. Dong, W. Matusik, M. Ding, W. Ding, P. Luo, and M. Tomizuka. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026a. doi: 10.48550/arXiv.2607.04434. URL https://arxiv.org/abs/2607.04434.

Y. Chen, Z. Bai, Z. Cao, W. Zeng, K. Q. Lin, Y. Lin, G. Liang, K. Y. Ma, Q. Huang, and M. Z. Shou. Show-harness: Just a VLM agent can play robots. *arXiv preprint arXiv:2609.10522*, 2026b. doi: 10.48550/arXiv.2609.10522. URL https://arxiv.org/abs/2609.10522.

Z. Chu, E. T. Barr, C. Le Goues, P. O'Hearn, M. Harman, F. Sarro, and H. Ye. OpenRUA: Robot-use agents are zero-shot visuomotor policies. *arXiv preprint arXiv:2610.02459*, 2026. doi: 10.48550/arXiv.2610.02459. URL https://arxiv.org/abs/2610.02459.

DeepSeek. Models and pricing. DeepSeek API Documentation, 2026. URL https://api-docs.deepseek.com/zh-cn/quick_start/pricing/. Accessed October 25, 2026.

Dream Labs. Zero-shot to new robots: Introducing dream-1, 2026. URL https://dreamlabs.ai/research/dream-1.

L. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, D. Niu, F.-F. Li, et al. Cap-x: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026.

Gemini Robotics Team, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, et al. Gemini robotics: Bringing AI into the physical world. *arXiv preprint arXiv:2503.20020*, 2025. doi: 10.48550/arXiv.2503.20020. URL https://arxiv.org/abs/2503.20020.

M.-H. Guo, Z.-H. Mo, J.-J. Wang, Y. Zhang, K. Wang, Y.-X. Deng, J.-P. Zhang, Y. Rao, and S.-M. Hu. Transferring the intelligence of VLMs to robotic control. *arXiv preprint arXiv:2609.22966*, 2026. doi: 10.48550/arXiv.2609.22966. URL https://arxiv.org/abs/2609.22966.

W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022. doi: 10.48550/arXiv.2207.05608. URL https://arxiv.org/abs/2207.05608.

W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023. doi: 10.48550/arXiv.2307.05973. URL https://arxiv.org/abs/2307.05973.

P. Isola. Robot-use agents, Sept. 2026. URL https://web.mit.edu/phillipi/www/writing/robot-use-agents.html. September 7, 2026.

N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024. doi: 10.48550/arXiv.2403.07974. URL https://arxiv.org/abs/2403.07974.

M. Jia, Y. Lin, X. Zhang, Z. Zhang, X. Liu, and M. Jiang. Agent as policy for robotic manipulation. *arXiv preprint arXiv:2609.12541*, 2026.

M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. doi: 10.48550/arXiv.2406.09246. URL https://arxiv.org/abs/2406.09246.

M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, Y. Ge, G. Lam, P. Liang, S. Song, M.-Y. Liu, C. Finn, and J. Gu. Cosmos Policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026. doi: 10.48550/arXiv.2601.16163. URL https://arxiv.org/abs/2601.16163.

K. Li, Z. Meng, H. Lin, Z. Luo, Y. Tian, J. Ma, Z. Huang, and T.-S. Chua. ScreenSpot-Pro: GUI grounding for professional high-resolution computer use. *arXiv preprint arXiv:2504.07981*, 2025. doi: 10.48550/arXiv.2504.07981. URL https://arxiv.org/abs/2504.07981.

Z. Li, Y. Zhang, W. Li, H. Huang, C. Wang, S. Deng, Y. Wei, T. Zhang, B. Xie, B. Zhou, Y. Chang, K. Zhou, Y.-C. Chen, J. Cheng, and Y. Li. Robo-Harness K1: Harnessing robot-use agents via perception augmentation. *arXiv preprint arXiv:2609.29389*, 2026. doi: 10.48550/arXiv.2609.29389. URL https://arxiv.org/abs/2609.29389.

J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022. doi: 10.48550/arXiv.2209.07753. URL https://arxiv.org/abs/2209.07753.

R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, et al. Aspire: Agentic/skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.

OpenAI. Codex cli: Lightweight coding agent that runs in the terminal. https://github.com/openai/codex, Oct. 2026a.

OpenAI. GPT-6 Astra: A new generation of intelligence, 2026b. URL https://openai.com/index/gpt-6-astra/. Accessed October 20, 2026.

OpenAI. GPT-6 Astra model: API pricing, 2026c. URL https://developers.openai.com/api/docs/models/gpt-6-astra. Accessed October 25, 2026.

Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. doi: 10.48550/arXiv.2504.16054. URL https://arxiv.org/abs/2504.16054.

B. Qu, W. Chen, and B. Zhang. RoboHarness: A simple harness outperforms VLA and world action models. *arXiv preprint arXiv:2610.14018*, 2026. doi: 10.48550/arXiv.2610.14018. URL https://arxiv.org/abs/2610.14018.

RoboDojo Team. RoboDojo simulation leaderboard, 2026. URL https://robodojo-benchmark.com/leaderboard. Accessed October 20, 2026.

R. Si, J. Bi, S. Yang, R. Ni, W. Huang, Q. Wang, S. Jiang, Y. Zhang, F. Ruan, K. Zhang, D. Wang, X. Li, H. Feng, Z. Dong, and D. Zhou. Fewer tokens, better action: Gpt-6 astra robot agents with 14% higher success rate but 65% fewer tokens. *arXiv preprint arXiv:2610.01939*, 2026.

J. Su, Y. Zheng, M. Yan, L. Yi, Z. Zhang, and H. Wang. GPT 6 Astra as an embodied policy. Technical report and code, 2026. URL https://github.com/anonymous-report-421/GPT-as-Policy. Accessed October 25, 2026.

H. Wang, H. Zou, H. Song, J. Feng, J. Fang, J. Lu, L. Liu, Q. Luo, et al. UI-TARS-2 technical report: Advancing GUI agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025. doi: 10.48550/arXiv.2509.02544. URL https://arxiv.org/abs/2509.02544.

J. Wang, Y. Yu, B. Zhu, Z. Yang, and J. Chen. SpatialHarness: Test-time spatial scaffolding for fine robotic manipulation. *arXiv preprint arXiv:2610.12457*, 2026. doi: 10.48550/arXiv.2610.12457. URL https://arxiv.org/abs/2610.12457.

Y. Yang, S. Chen, J. Gao, and H. Zhang. ORA: Open robot-use agents. Software release, Oct. 2026. URL https://github.com/hao-ai-lab/ORA. Released October 15, 2026. Accessed October 20, 2026.

S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. doi: 10.48550/arXiv.2602.15922. URL https://arxiv.org/abs/2602.15922.

X. Yue, T. Zheng, Y. Ni, Y. Wang, K. Zhang, S. Tong, Y. Sun, B. Yu, G. Zhang, H. Sun, Y. Su, W. Chen, and G. Neubig. MMMU-Pro: A more robust multi-discipline multimodal understanding benchmark. *arXiv preprint arXiv:2409.02813*, 2024. doi: 10.48550/arXiv.2409.02813. URL https://arxiv.org/abs/2409.02813.

W. Zhang, K. Wang, Y. Ouyang, X. Huang, L. Li, K. Su, W. Jin, W. Chai, H. Liang, Z. Dou, Y. Chen, and T. Chen. An unexpected robot policy: Early evaluations of GPT-6 Astra on RoboDojo and beyond. *arXiv preprint arXiv:2609.24170*, 2026a. doi: 10.48550/arXiv.2609.24170. URL https://arxiv.org/abs/2609.24170.

Y. Zhang, H. Huang, Y. Chang, J. Su, B. Zhou, Y. Xu, W. Chen, T. Zhou, C. Wang, T. Zhang, Y. Wei, W. Li, S. Deng, Y. Li, Y.-C. Chen, and Z. Li. World action agent: Harnessing VLMs for robot manipulation via world action rehearsal. *arXiv preprint arXiv:2609.29964*, 2026b. doi: 10.48550/arXiv.2609.29964. URL https://arxiv.org/abs/2609.29964.

Y. Zhang, H. Zhang, F. Gao, X. Li, Z. Liu, Y. Nie, C. Zhu, J. Qiu, Y. Yan, J. Liu, W. Tang, J. Rao, Z. Fang, C. Wei, Y. Wang, W. Ding, and C. Yu. Harness VLA: Steering frozen VLAs into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026c. doi: 10.48550/arXiv.2607.08448. URL https://arxiv.org/abs/2607.08448.

A. Experimental Setup and Reproducibility Settings

Simulation benchmark and baselines. RoboDojo provides 42 tasks in Isaac Sim (Chen et al., 2026a). We compare agent harnesses using Qwen3.8-Flash-Next as the planner. GPT-as-Policy (GAP) (Su et al., 2026) uses its hybrid configuration with $\pi_{0.5}$; RoboProbe (Zhang et al., 2026a) and RPent (Zhang et al., 2026c) use their default configurations. Table 1 also includes the published Astra–RoboProbe results and representative vision-language-action (VLA) and world-action models (WAM) from the RoboDojo leaderboard (RoboDojo Team, 2026).

Reasoning effort. We use `medium` reasoning effort for GPT-6 Astra, `high` for both GPT-5.6 Sol and GPT-6.1 Sol, `xhigh` for all Qwen-series Models, and `max` for DeepSeek-V4.1-Flash.

B. Real Eval Benchmark Details

Real Eval evaluates tabletop manipulation on a dual-arm gripper platform through five capability dimensions: *spatial understanding*, *object generalization*, *instruction following*, *compositional generalization*, and *unseen tasks*. Its test catalogue contains 19 evaluation entries, with 20 points allocated to each dimension and 100 points in total. An evaluation entry specifies an operation together with the variation being tested: bowl stacking, for example, appears both as an object-generalization test and as a direction-following test. The catalogue therefore distinguishes changes in objects, language, and task structure even when they share a manipulation primitive. Table 3 gives the complete test catalogue and its scoring criteria.

B.1. Hardware and Baselines

All methods run on an ARX AC one dual-arm platform with two X5 arms and three Intel RealSense D405 cameras, comprising a head view and a wrist view on each arm. We evaluate uni-robot-agent with GPT-6 Astra and Qwen3.8-Flash-Next as planners and compare both configurations with $\pi_{0.5}$ (Physical Intelligence et al., 2025), fine-tuned on demonstrations from the same platform. We also include Dream-1 (Dream Labs, 2026), a world action model evaluated zero-shot through its public API (`dreamlabs/dream-1-beta`). In the real-world evaluation, uni-robot-agent additionally uses the D405 depth stream, whereas $\pi_{0.5}$ and Dream-1 natively consume only RGB images.

B.2. Training Task Catalogue and Collection Protocol

The accompanying training catalogue lists 30 task templates and 1,550 trajectories: 27 templates specify 50 trajectories each, while bag zipping, coin insertion, and tissue extraction specify 100, 80, and 20, respectively. The templates span stacking and sorting, object transfer and handover, container loading, cloth manipulation, surface wiping, and operation of buttons, switches, drawers, and a microwave door. Collection protocols vary object placement, material, color, container arrangement, and table appearance at task-specific intervals. For example, paper-cup stacking uses four to six inverted cups at different positions, changes cup material every five trajectories, and changes the table mat every ten. Test-tube transfer uses three to four tubes of different sizes and reverses the transfer direction every five trajectories. Cloth folding includes two folds per trajectory, changes cloth orientation every three trajectories, and changes the cloth every six. These collection settings establish the reference operations and variations against which the test catalogue defines new objects, instructions, and action

combinations. The catalogue is collected to fine-tune end-to-end embodied policies such as $\pi_{0.5}$, whereas uni-robot-agent does not use any of these demonstrations; for uni-robot-agent, every entry is therefore evaluated zero-shot, and the five dimensions characterize the type of variation that each entry introduces.

B.3. Evaluation Dimensions

Spatial understanding. This dimension includes one simple entry (fruit transfer), two intermediate entries (cloth folding and plate extraction), and two complex entries (test-tube transfer and mug hanging). Fruit transfer moves cherries, bananas, apples, oranges, and melons from bowls or plates into a basket. Cloth folding varies size and orientation. Plate extraction requires one arm to lift a bottle while the other removes and stacks the plate beneath it. Test-tube transfer and mug hanging further distinguish acquiring an object from completing the subsequent placement.

Object generalization. The operation remains bottle handover or bowl stacking while the manipulated objects change. Handover introduces tea, coconut-water, and bottled-water containers, together with handled stainless-steel and plastic cups. Bowl stacking introduces stainless-steel bowls, small and large paper bowls, ceramic bowls, and large salad bowls. These variations probe differences in shape, size, material, and available grasp geometry.

Instruction following. Three tests separate color, object, and spatial-language grounding. Color-conditioned cup stacking places two or three differently colored cups on the table and specifies which color should be stacked onto which other color. Object-conditioned pick-and-place uses a toy car, five fruit types, blocks, and three tableware types, with bowls, plates, and baskets as target containers. Direction-conditioned bowl stacking requests left-to-right or right-to-left arrangements. These tests evaluate whether a physically feasible operation is directed toward the requested object, destination, and spatial relation.

Compositional generalization. Four entries recombine familiar operations, objects, and semantic constraints. Tissue extraction followed by disposal chains two actions into one objective. Stacking upward-facing cups changes the object configuration relative to the inverted-cup training template. Wiping a whiteboard with a rag combines the wiping operation with an object used elsewhere in the task catalogue, replacing the whiteboard eraser specified for training. Arranging cubes by color changes the criterion from the size ordering specified in the training catalogue. Together, these cases test action sequencing, object substitution, and semantic recombination.

Unseen tasks. The final dimension organizes transfer by its relation to the training catalogue. Closing the microwave door isolates a sub-action of the microwave loading routine. Taking a bowl out of the microwave and unfolding a rag reverse the corresponding loading and folding operations. Stacking books is listed as a combined action, while building a three-level Tower of Hanoi is listed as a new action.

Table 3 | Complete Real Eval test catalogue. Each dimension has a 20-point budget, and score caps follow the source specification. IDs match Table 4.

ID	Task	Task variation and scoring criterion	Max
Spatial Understanding			**20**
1	Pick and place (fruits)	Move cherries, bananas, apples, oranges, and melons from bowls or plates into a basket. One point for each fruit type placed successfully in every trial.	5
2	Fold rag	Fold large, small, and obliquely placed rags. One point for each condition folded successfully.	3
3	Bimanual plate pull-out	Lift the bottle with one arm and remove and stack the plate with the other. Each arm's required action receives 0.5 points per trial.	4
4	Insert test tubes into rack	Transfer test tubes from one rack and insert them into another. Each pickup receives 0.5 points and each insertion 0.5 points.	4
5	Hang mug	Hang a mug on a rack. Each grasp receives 0.5 points and each successful hang 0.5 points.	4
Object Generalization			**20**
6	Bimanual bottle handover	New beverage bottles and handled steel/plastic cups. One point per successful handover, capped at ten.	10
7	Stack bowls (new objects)	New steel, small/large paper, ceramic, and large salad bowls. One point per successful stack, capped at ten.	10
Instruction Following			**20**
8	Stack cups (color order)	Two- and three-color settings; one point per correctly distinguished color configuration, capped at five points for each setting.	10
9	Pick and place (assorted objects)	Select among a toy car, five fruits, blocks, and three tableware types; place into a bowl, plate, or basket. Each distinguished object and each container category receives 0.5 points.	6.5
10	Stack bowls (given direction)	Follow left/right and above/below relations. Left/right discrimination receives two points; above/below receives 1.5.	3.5
Compositional Generalization			**20**
11	Pull out tissue and discard	Pull a tissue out of its pack and throw it into a bin. Each extraction and each disposal receives 0.5 points, up to two points each.	4
12	Stack upward-facing cups	Stack paper cups with their openings facing upward. One point per successful stack.	4
13	Wipe whiteboard	Use a rag to wipe the whiteboard. One point per successful wipe.	4
14	Arrange cubes (color order)	Arrange cubes according to color in two- and three-color settings; one point per correctly distinguished configuration, capped at four points per setting.	8
Unseen Tasks			**20**
15	Close microwave door	Sub-action transfer: close the microwave door. One point per success.	4

Continued on the next page.

Table 3 | Real Eval test catalogue (continued).

ID	Task	Task variation and scoring criterion	Max
16	Take bowl from microwave	Inverse action: take a bowl out of the microwave. One point per success.	4
17	Unfold rag	Inverse action: unfold the rag. One point per success.	4
18	Stack books	Combined action: stack notebooks. One point per success.	4
19	Tower of Hanoi	New action: build a three-level Tower of Hanoi. One point per success.	4

B.4. Per-Task Results

Table 4 reports the individual task scores underlying Table 2, using the task IDs and scoring criteria above.

B.5. Scoring Protocol

Before each trial, the evaluator resets the scene and randomizes object placements and goal regions. Each task provides a pool of paraphrased instructions; the evaluator selects one per trial and passes it to the method as the only task specification. All methods, including $\pi_{0.5}$ and uni-robot-agent, receive the same task-level instruction, without task-specific demonstrations, subtask plans, or hints. After each trial, the evaluator records whether each scoring criterion is satisfied, and all methods are scored with the same rubric.

Let $\mathcal{T}_d$ denote the evaluation entries in dimension d, and let p_t and m_t be the awarded points and stated maximum for entry t. The benchmark's score allocation is

$$S_d = \sum_{t \in \mathcal{T}_d} p_t, \qquad \sum_{t \in \mathcal{T}_d} m_t = 20, \qquad \text{Score}_{\text{Real Eval}} = \sum_{d=1}^{5} S_d, \tag{1}$$

with $0 \le p_t \le m_t$ and a total score on a 0–100 scale. Equal dimension budgets keep all five capabilities represented, while the individual task weights follow the source rubric. Instruction tests additionally award credit for correctly distinguished colors, objects, containers, or directions. Thus, the aggregate is a rubric-based capability score, rather than the fraction of episodes that fully complete their objectives. In particular, successful grasping followed by failed insertion can receive partial credit without constituting complete task success.

C. API Cost Estimation

Fewer tokens do not necessarily mean lower cost: models differ in their prices for input, cached input, and output. For the comparisons in Section 6.1, we estimate API cost by applying the public rates in Table 5 to each request's recorded usage and summing the charges over an episode. Uncached and cached input are billed separately. Reasoning tokens are already included in output usage and are counted only once. Recorded retries are included; requests without recoverable usage are left unestimated.

Astra. Long-context requests and cache writes require additional pricing rules. Requests exceeding 272K input tokens are charged at twice the input and cache rates and 1.5 times the output rate. For recorded cache writes, we use the 30-minute rate of USD 12.50 per million tokens, also applying this rate when the retention duration is unspecified.

Table 4 | **Per-task results on Real Eval.** Each capability subtotal is out of 20, and Total is out of 100. Task IDs correspond to Table 3; scoring details are given in Appendix B.5.

			VLA	WAM	uni-robot-agent	
ID	**Task**	**Max**	$\pi_{0.5}$	Dream-1	GPT-6 Astra	Qwen3.8-Flash-Next
Spatial Understanding						
1	Pick and place (fruits)	5	2	3	5	3
2	Fold rag	3	1.75	0	3	1
3	Bimanual plate pull-out	4	3	0	2	1
4	Insert test tubes into rack	4	2	1	3	2
5	Hang mug	4	2.5	0	4	2
	Subtotal	*20*	*11.25*	*4.00*	*17.00*	*9.00*
Object Generalization						
6	Bimanual bottle handover	10	2.5	0	5	0
7	Stack bowls (new objects)	10	10	8	7.5	6
	Subtotal	*20*	*12.50*	*8.00*	*12.50*	*6.00*
Instruction Following						
8	Stack cups (color order)	10	6	1	8	5
9	Pick and place (assorted objects)	6.5	6	6.5	6.5	6.5
10	Stack bowls (given direction)	3.5	2	2	3.5	2
	Subtotal	*20*	*14.00*	*9.50*	*18.00*	*13.50*
Compositional Generalization						
11	Pull out tissue and discard	4	0	4	4	1
12	Stack upward-facing cups	4	4	0	4	0
13	Wipe whiteboard	4	2	4	4	4
14	Arrange cubes (color order)	8	0	2	8	8
	Subtotal	*20*	*6.00*	*10.00*	*20.00*	*13.00*
Unseen Tasks						
15	Close microwave door	4	4	4	4	4
16	Take bowl from microwave	4	0	0	0	0
17	Unfold rag	4	0	0	0	0
18	Stack books	4	0	0	2	4
19	Tower of Hanoi	4	0	0	2	0
	Subtotal	*20*	*4.00*	*4.00*	*8.00*	*8.00*
	Total↑	**100**	47.75	35.50	**75.50**	49.50

DeepSeek. The applicable rate depends on when a request is processed. We use the recorded response time, falling back to the request start time if it is unavailable. Peak rates apply from 09:00–12:00 and 14:00–18:00 on working days in the Asia/Shanghai time zone; all other times use off-peak rates. None of the evaluated requests fall on public holidays.

D. Monitored Motion Execution

The execution monitoring described in Section 2.5 addresses a basic difficulty in robot control: issuing a motion command does not establish that the robot has reached its target. Planning may reject a command, or execution may stop making progress. The `move` tool therefore advances through bounded waypoints, checks the state returned after each step, and reports why execution ended so that the agent can decide what to do next.

Table 5 | **Reference API rates for the cost analysis.** Rates are per million tokens in the listed currency.

Model / pricing condition	Currency	Input	Cached input	Output
GPT-6 Astra (OpenAI, 2026c)	USD	10.00	1.00	50.00
Qwen3.8-Flash (Alibaba Cloud, 2026)	CNY	0.80	0.10	2.70
DeepSeek-V4.1-Flash (peak) (DeepSeek, 2026)	CNY	2.00	0.04	8.00
DeepSeek-V4.1-Flash (off-peak)	CNY	1.00	0.02	4.00

Rates were archived on October 22–23, 2026. Qwen uses the Beijing-region `qwen3.8-flash` rates. All costs are reported in USD, with CNY prices converted at 6.6729 CNY per USD.

Algorithm 1 gives this loop for a single active arm without a gripper-opening target. We denote the requested pose by $g = (p_g, q_g)$ and the execution target by $\tilde{g}$, initially equal to g. The two are kept separate because a supported planning rejection may lead to an alternative execution target within the original tolerances. Other arms retain their initial pose and gripper commands. For multi-arm requests, each active arm has its own pose tracker and all target poses must be reached; optional gripper-opening checks are omitted from the pseudocode.

Algorithm 1 Monitored execution of a pose-only `move` request

Require: Target g, tolerances (ϵ_p, ϵ_q), waypoint limit δ, rotation radius r, accepted-step budget B
Ensure: Actual pose, target errors, arrival flag, step count, stop reason, and diagnostic evidence

$s \leftarrow$ READSTATE; $\tilde{g} \leftarrow g$; $k \leftarrow 0$; $a \leftarrow$ false
Initialize progress history H from s and $\tilde{g}$; $z \leftarrow$ `max_steps`
while $k < B$ **do**
  if s has ended **then** $z \leftarrow$ `native_end`; **break**
  $w \leftarrow$ WAYPOINT$(s, \tilde{g}, \delta, r)$
  $(s', b, d) \leftarrow$ STEP(w) ▷ New state, acceptance flag, diagnostics
  $u \leftarrow$ ABNORMALMOTION(s, s'); $s \leftarrow s'$
  if b **then** $k \leftarrow k + 1$
  if s has ended **then** $z \leftarrow$ `native_end`; **break**
  if u **then** $z \leftarrow$ `unsafe_motion`; retain motion evidence; **break**
  if not b **then**
    if not a **and** ELIGIBLEGOALSEARCH(d) **then**
      $a \leftarrow$ true
      $g' \leftarrow$ SOLVEGOALREGION$(s, g, \epsilon_p, \epsilon_q)$
      if a valid alternative goal g' is found **then**
        $\tilde{g} \leftarrow g'$; reset H using s and $\tilde{g}$
        continue
      end if
    end if
    $z \leftarrow$ `planning_rejected`; retain rejection evidence; **break**
  end if
  Update H with position and orientation errors relative to $\tilde{g}$
  if REACHED$(s, g, \epsilon_p, \epsilon_q)$ **then**
    $z \leftarrow$ `pose_reached`; **break**
  else if DIVERGING(H) **then**
    $z \leftarrow$ `diverging`; retain progress evidence; **break**
  else if STALLED$(H, \epsilon_p, \epsilon_q)$ **then**
    $z \leftarrow$ `stalled`; retain progress evidence; **break**
  end if
end while
return FEEDBACK$(s, g, \tilde{g}, k, z)$ with diagnostic evidence

Waypoint generation. Each waypoint is computed from the latest measured pose, so the next command accounts for where the robot actually moved. Translation and rotation share a displacement budget δ, with the arm's declared rotation radius r used to account for displacement due to rotation. The angular step is capped at

$$\theta_{\max} = \begin{cases} \min\{0.6,\, 2\arcsin(\min\{1, \delta/(4r)\})\}, & r > 0, \\ 0.6, & r = 0, \end{cases} \tag{2}$$

in radians. The orientation is interpolated toward $\tilde{g}$ within this limit. For the resulting rotation $\Delta\theta$, the remaining translation allowance is

$$\delta_p = \max\{0,\, \delta - 2r\sin(\Delta\theta/2)\}. \tag{3}$$

The position advances toward $\tilde{g}$ by at most δ_p.

Stopping criteria. The tool checks both whether the requested pose has been reached and whether continued execution is making progress. Arrival requires position and orientation errors relative to g to fall within ϵ_p and ϵ_q. Position error is Euclidean distance; orientation error is the shortest angular distance between unit quaternions,

$$d_q(q_1, q_2) = 2\arccos(\min\{1, |q_1^\top q_2|\}). \tag{4}$$

Progress is measured toward the execution target $\tilde{g}$. Let e_t^p and e_t^q be the corresponding errors after accepted step t. To detect motion away from the target, DIVERGING tracks $v_t = e_t^p + 0.1e_t^q$, with position in meters and orientation in radians, and stops after two consecutive increases satisfying $v_t > v_{t-1} + 0.01$. To detect lack of progress, STALLED examines four accepted-step measurements and stops when neither error has improved sufficiently:

$$e_{t-3}^p - \min_{j=t-2}^{t} e_j^p \le 0.1\epsilon_p, \qquad e_{t-3}^q - \min_{j=t-2}^{t} e_j^q \le 0.1\epsilon_q. \tag{5}$$

Large changes in measured pose are checked separately by ABNORMALMOTION. The thresholds are 0.2 m or 0.8 rad for an active arm, and 0.05 m or 0.25 rad for an inactive arm. These checks also apply to rejected commands: rejection does not guarantee that the returned state is unchanged. Execution also stops when the environment ends or the step budget B is exhausted. Only accepted steps count toward B or update the progress history.

Planning rejections and feedback. A planning rejection may allow the tool to try a nearby execution target without relaxing the agent's original request. On backends that support goal-region search, ELIGIBLEGOALSEARCH permits one search per `move` call following an active-arm rejection of type `ik_failed`, `joint_jump_rejected`, or `ik_candidate_rejected`. An alternative goal is accepted only if it refers to the current state version and lies within the original position and orientation tolerances. It replaces $\tilde{g}$ and resets the progress history, while arrival continues to be checked against g.

The returned feedback tells the agent where the robot ended up, how far it remains from the requested pose, and why execution stopped. It includes actual and target poses, position and orientation errors, arrival status, accepted-step count, and available rejection or recovery diagnostics. Reaching a pose does not by itself establish that a grasp, placement, or other task objective has succeeded.

E. Spatial Grounding Through Agent-Written Code

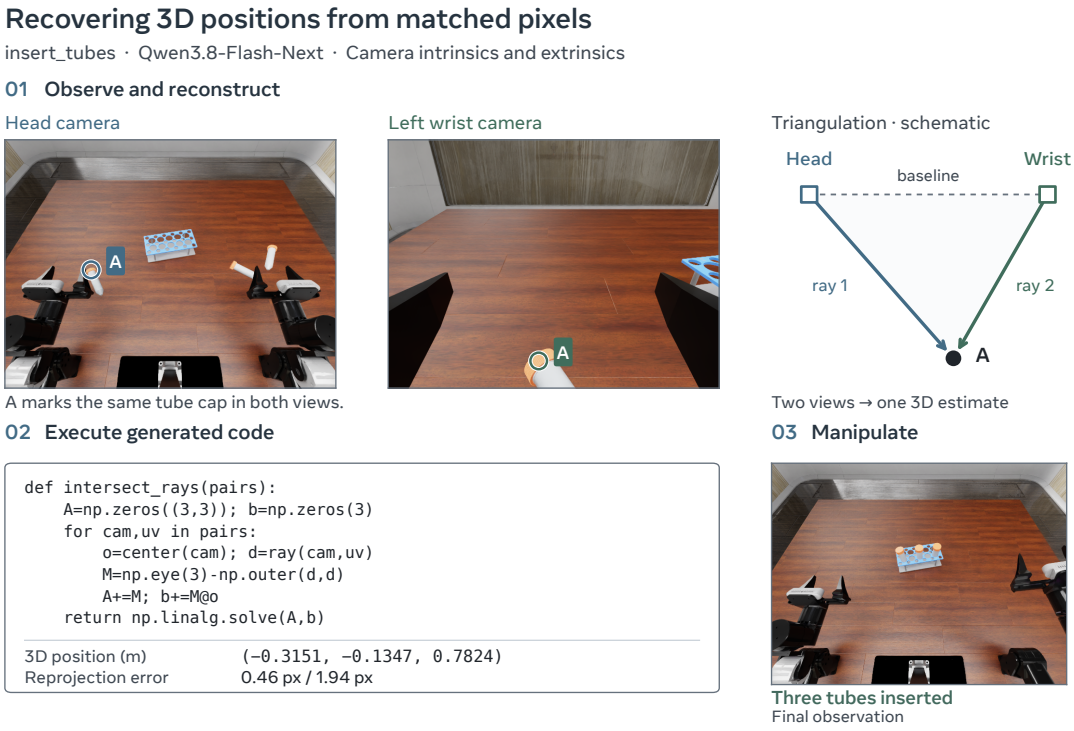


Figure 15 | **Agent-written triangulation in `insert_tubes`.** Matching pixels marked A define viewing rays, illustrated schematically above the executed code excerpt. Recorded coordinates and reprojection errors connect the calculation to its output; the final frame follows subsequent grasping and insertion.

The depth ablation in Section 5.1 raises a practical question: how does an agent obtain usable spatial coordinates when its observations contain no depth? We examine two successful Qwen3.8-Flash-Next episodes from the no-depth, no-skill configuration. In both, the agent writes and executes geometry code using RGB observations and the supplied camera intrinsics and extrinsics, rather than calling the built-in triangulation operation. The cases illustrate different ways of constructing localization estimates: matching features across views and testing geometric assumptions against image evidence.

E.1. Recovering Tube Positions Across Views

In `insert_tubes`, the agent must place three tubes into a rack. After obtaining image-space segmentation boxes, it captures additional camera views and writes a Python function to intersect viewing rays in a least-squares sense. It then selects corresponding tube-cap pixels and computes their world coordinates. Figure 15 follows one cap: the calculated position reprojects within 0.46 and 1.94 pixels of the selected points in the two views. This gives the agent a way to check whether its chosen correspondences agree geometrically.

The cap positions are only the beginning of the manipulation sequence. The agent subsequently estimates tube axes, chooses grasp poses, and uses observations and action feedback to guide lifting and insertion. The episode ends with all three tubes in the rack. Here, code execution turns image correspondences into coordinates that the agent can inspect and use in its action planning.

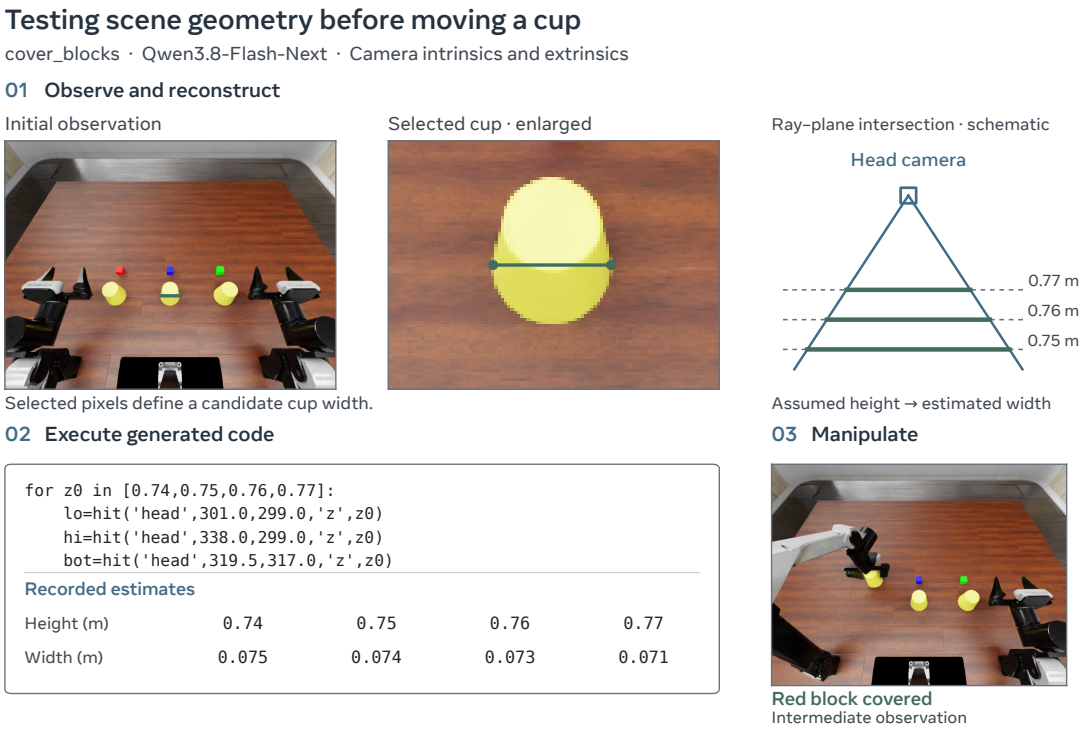


Figure 16 | **Geometric hypothesis testing in `cover_blocks`.** The agent varies a candidate table height and computes cup dimensions from selected pixels. The schematic illustrates the ray–plane intersections computed by `hit()` in the code excerpt; the right-hand frame shows a later placement over the red block.

E.2. Exploring Geometry Before Moving a Cup

In `cover_blocks`, the agent must cover three colored blocks from left to right, then uncover them in a specified color order. Locating the cups requires reasoning about their extent as well as their image centers. The agent extracts colored image regions and writes code that tests candidate table heights, intersects pixel rays with each candidate plane, and compares the resulting cup dimensions (Figure 16). The height sweep exposes how an assumed scene parameter changes the inferred geometry, rather than treating a single image estimate as fixed.

The agent continues by projecting candidate object shapes into the images and refining its scene estimates. It then uses the estimated cup positions to approach, grasp, and place the cups, checking the scene as the task proceeds. The recorded episode completes the required covering and uncovering sequence. This case shows a more exploratory use of code: the agent constructs and tests geometric hypotheses before committing to physical actions. Together, the two cases illustrate how RGB-based localization can support successful execution, while requiring correspondence selection, geometric assumptions, and verification that depth access can simplify.

F. Planning During Physical Execution

The overlap measurements in Section 6.2 show when model requests and physical actions run concurrently, but not what the agent prepares during that time or how it uses the result. We follow longer excerpts from two recorded trajectories to connect this preparation to subsequent actions and feedback. Each figure pairs key observations with the recorded execution timeline

and enlarges one interval in which the model generates a concrete calculation while a physical tool is running. The annotations summarize recorded outputs; the bars denote model requests, not a measurement of internal reasoning time.

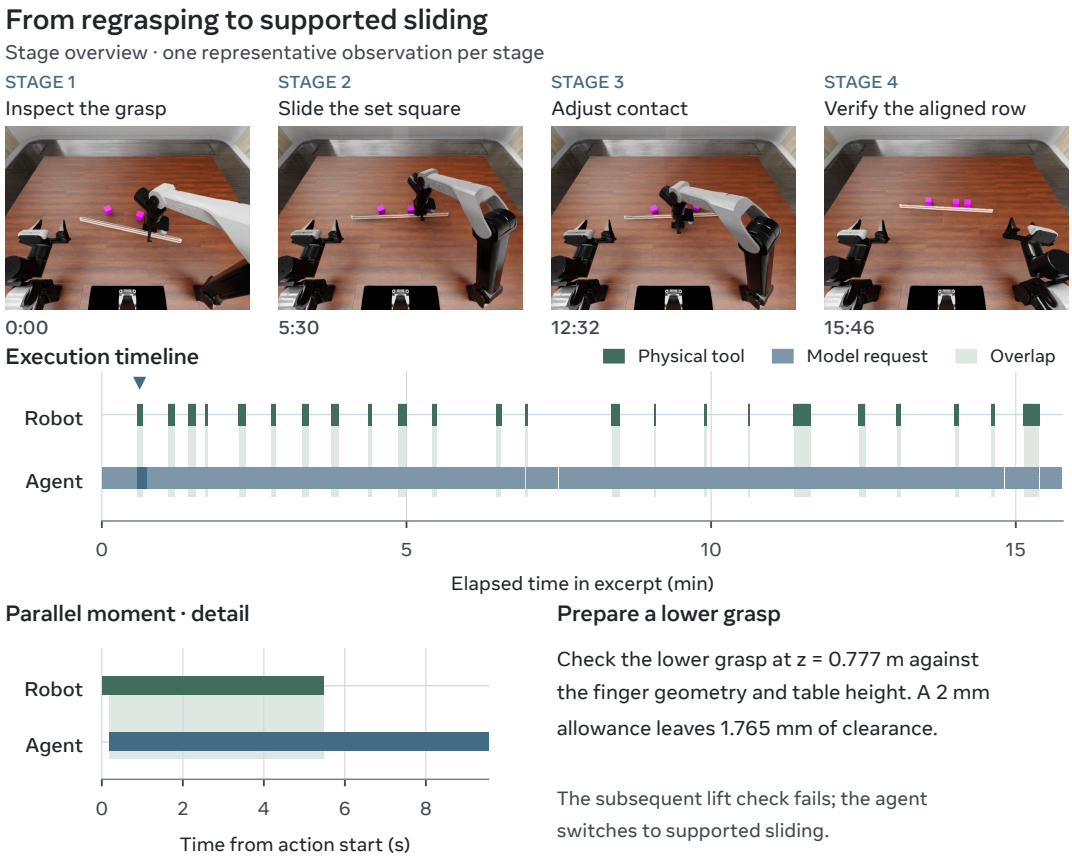


Figure 17 | **Grasp adjustment and supported sliding in `align_blocks`.** The upper row summarizes four stages, each illustrated by a representative observation; a stage may contain multiple tool calls. Green and blue bars show physical tools and model requests; shading marks overlap. The marked interval is enlarged below, where the model prepares a clearance check during release. Image timestamps and the main axis are relative to the first displayed observation.

F.1. From Regrasping to Supported Sliding

In `align_blocks`, the agent must align three blocks with a set square while keeping the blocks on the table. Figure 17 follows the trajectory from an uncertain grasp through contact adjustments to the final aligned row. While a background `release` opens the gripper, the model generates a Python check for a lower grasp pose. Using the finger geometry and table height, it checks that the proposed pose leaves clearance even after a 2 mm uncertainty allowance. The program runs after the release finishes, and the agent then moves to the checked pose, grasps, and tests retention with a short lift.

The lift does not retain the set square. The agent explicitly changes its approach: it will keep the square supported and use sliding contact instead. It lowers the hand, closes the gripper, and issues a sequence of small rotations and translations, then inspects the scene and adjusts contact again. The final observation shows the blocks in a row with the arms clear. This longer sequence

shows both the use of preparation during background execution and the need to revise that preparation when subsequent physical evidence contradicts it.

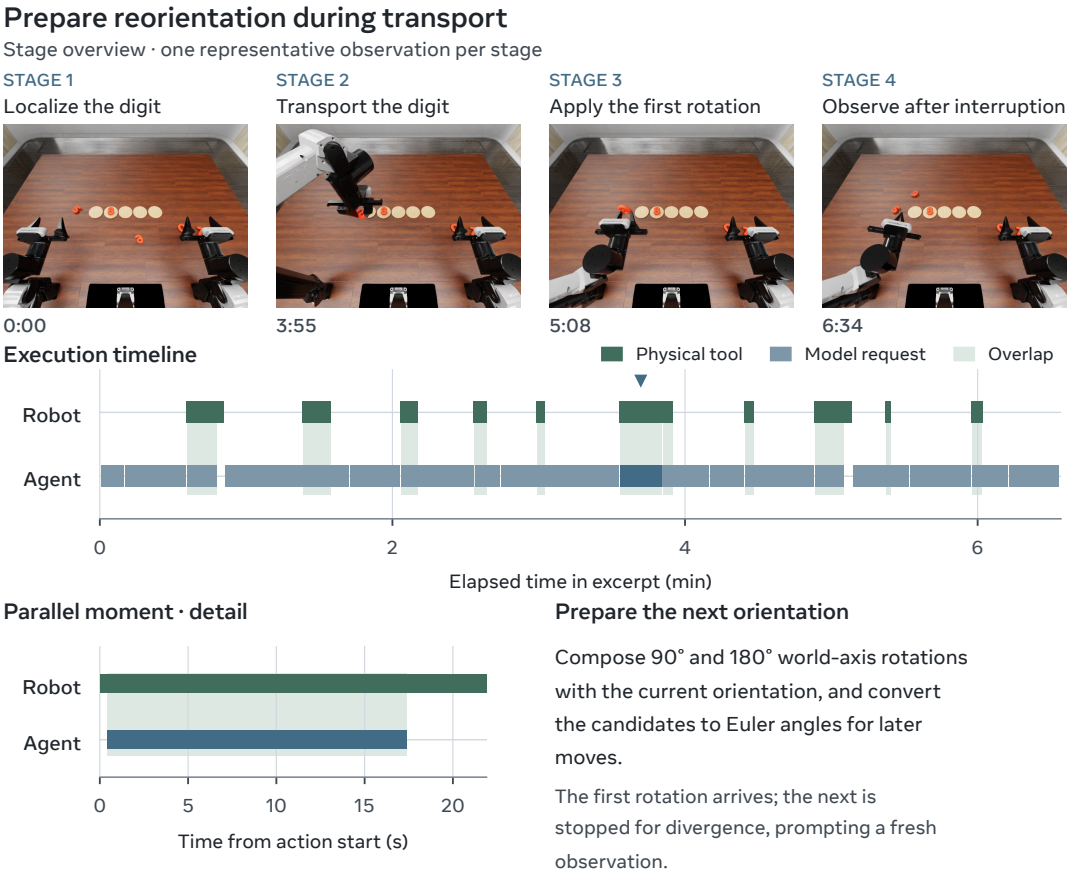


Figure 18 | **Orientation preparation during digit transport.** The upper row summarizes four stages rather than individual tool calls; the lanes below show recorded execution. The enlarged interval shows a model request preparing rotation candidates while transport continues. The later divergence is retained to show how execution feedback affects the prepared plan. Colors and time conventions match Figure 17.

F.2. Preparing Reorientation During Transport

In `arrange_largest_number`, the agent must place digit-shaped objects on pads to form the largest number. The excerpt in Figure 18 follows one digit from localization and grasping through transport and reorientation. While the arm moves toward a raised target, the model generates a program that composes candidate 90° and 180° world-axis rotations with the current orientation and converts them to Euler angles. This prepares later orientation commands without waiting for the transport result.

The transport first exhausts its step budget, so the agent continues toward the same target before attempting the prepared rotation. The first orientation target is reached, but the next rotation stops for divergence. The agent then raises the hand while retaining its measured orientation and captures a new observation. The sequence illustrates why advance preparation and execution monitoring are complementary: a candidate action can be prepared during an earlier movement, while its application still depends on the state returned by the robot.